



AirPredict:AI-Powered Healthy Indoor Environment

Project Report

Administrator

fadia.m.mahmoud@gmail.com

ABSTRACT

Indoor air quality (IAQ) has a direct impact on human health, particularly for individuals suffering from respiratory conditions such as asthma or sensitivity to enclosed spaces. Poor indoor air conditions caused by high carbon dioxide levels, inadequate ventilation, and unfavorable thermal conditions can lead to discomfort and serious health risks.

This project proposes an intelligent system that predicts indoor air quality using machine learning techniques based on environmental and contextual features, including carbon dioxide concentration, temperature, relative humidity, occupancy level, and window status. The IAQ index is used as a reference standard to label air quality levels into predefined categories. Several machine learning and deep learning models are trained to learn the relationship between these factors and the resulting air quality condition.

Based on the predicted air quality level, the system provides real-time, user-centered recommendations to improve indoor conditions, such as increasing ventilation or reducing occupancy. The proposed approach demonstrates how artificial intelligence can be effectively applied to enhance indoor environmental quality and support health-aware decision-making in enclosed spaces.

TABLE OF CONTENTS

ABSTRAC	1
TABLE OF CONTENTS	11
ABBREVIATIONS.....	111
LIST OF FIGURES.....	1111
LIST OF Tables	11111
Chapter 1:	Introduction
.....	10
1.1 Overview.....	10
1.2 Project Motivation	10
1.3 Problem Statement.....	11
1.4 Project Aim and Objectives	11
1.5 Project Limitations (This section can be moved to chapter 7)	Error!
Bookmark not defined.	
1.6 Project Expected Output	Error! Bookmark not defined.
1.7 Project Schedule	Error! Bookmark not defined.
1.8 Report Organization.....	Error! Bookmark not defined.
Chapter 2:	Literature Review
.....	Error! Bookmark not defined.
2.1 Existing Systems.....	Error! Bookmark not defined.
2.2 Limitations of Existing Systems.....	Error! Bookmark not defined.
2.3 Overall Solution Approach	Error! Bookmark not defined.

Chapter 3: Requirement Engineering and Analysis

..... **Error! Bookmark not defined.**

3.1 Data Understanding and Preparation **Error! Bookmark not defined.**

3.1.1 Data Collection Methods and Sources **Error! Bookmark not defined.**

3.1.2 Data Preprocessing Techniques and Cleaning **Error! Bookmark not defined.**

3.1.3 Data Integration and Transformation... **Error! Bookmark not defined.**

3.2 Feature Engineering and Selection **Error! Bookmark not defined.**

3.3 Ethical Considerations and Data Privacy ... **Error! Bookmark not defined.**

Chapter 4: Model Development and Architectural Design **Error! Bookmark not defined.**

4.1 System Architecture Overview **Error! Bookmark not defined.**

4.2 Data Pipeline Design **Error! Bookmark not defined.**

4.3 Model Architecture and Design..... **Error! Bookmark not defined.**

4.4 Training Process **Error! Bookmark not defined.**

4.5 Hyperparameter Tuning..... **Error! Bookmark not defined.**

4.6 Validation Strategy **Error! Bookmark not defined.**

4.7 Code Implementation (this section can be moved to the appendix).... **Error! Bookmark not defined.**

Chapter 5: Model/System Evaluation and Testing Plan **Error! Bookmark not defined.**

5.1 Testing Plan Overview **Error! Bookmark not defined.**

5.1.1 Unit Testing **Error! Bookmark not defined.**

5.1.2 Integration Testing **Error! Bookmark not defined.**

5.1.3 Performance Testing **Error! Bookmark not defined.**

5.2 Evaluation Metrics.....	Error! Bookmark not defined.
5.2.1 Model Performance Metrics.....	Error! Bookmark not defined.
5.2.2 Other Relevant Metrics	Error! Bookmark not defined.
5.3 Evaluation Methodology	Error! Bookmark not defined.
5.3.1 Cross-Validation Techniques	Error! Bookmark not defined.
5.3.2 Holdout Method	Error! Bookmark not defined.
5.3.3 Train-Test Splits.....	Error! Bookmark not defined.
5.4 Model Performance Evaluation	Error! Bookmark not defined.
5.4.1 Performance on Training Data	Error! Bookmark not defined.
5.4.2 Performance on Validation Data.....	Error! Bookmark not defined.
5.4.3 Performance on Test Data.....	Error! Bookmark not defined.
5.5 System Evaluation	Error! Bookmark not defined.
5.5.1 Scalability Assessment.....	Error! Bookmark not defined.
5.5.2 Resource Utilization.....	Error! Bookmark not defined.
5.5.3 Responsiveness and Latency.....	Error! Bookmark not defined.
5.6 Comparison and Discussion	Error! Bookmark not defined.

Chapter 6: Model Deployment and Integration

..... **Error! Bookmark not defined.**

6.1 Technology Stack for AI Systems or Robotics.....	Error! Bookmark not defined.
6.2 Data Acquisition and Preprocessing Pipeline.....	Error! Bookmark not defined.
6.3 Model Deployment Strategies	Error! Bookmark not defined.
6.4 Model Integration within AI Systems or Robotic.....	Error! Bookmark not defined.
6.5 User Interface Design (if applicable).....	Error! Bookmark not defined.

chapter 7: Conclusion and future workError!

Bookmark not defined.

7.1 Summary and Project Evaluation **Error! Bookmark not defined.**

7.2 Future Directions **Error! Bookmark not defined.**

References Error! Bookmark not defined.

ABBREVIATIONS

AI – ARTIFICIAL INTELLIGENCE

ML – MACHINE LEARNING

DL – DEEP LEARNING

IAQ – INDOOR AIR QUALITY

AQI – AIR QUALITY INDEX

CO₂ – CARBON DIOXIDE

CO – CARBON MONOXIDE

RH – RELATIVE HUMIDITY

VOC – VOLATILE ORGANIC COMPOUNDS

PM – PARTICULATE MATTER

WHO – WORLD HEALTH ORGANIZATION

ASHRAE – AMERICAN SOCIETY OF HEATING, REFRIGERATING AND AIR-
CONDITIONING ENGINEERS

IT – INFORMATION TECHNOLOGY

CNN – CONVOLUTIONAL NEURAL NETWORK

LSTM – LONG SHORT-TERM MEMORY

KNN – K-NEAREST NEIGHBORS

SVM – SUPPORT VECTOR MACHINE

MAE – MEAN ABSOLUTE ERROR

RMSE – ROOT MEAN SQUARE ERROR

MAPE – MEAN ABSOLUTE PERCENTAGE ERROR

CSV – COMMA-SEPARATED VALUES

API – APPLICATION PROGRAMMING INTERFACE

TP – TRUE POSITIVE

TN – TRUE NEGATIVE

FP – FALSE POSITIVE

FN – FALSE NEGATIVE

TC – TEST CASE

CPU – CENTRAL PROCESSING UNIT

HVAC – HEATING, VENTILATION, AND AIR CONDITIONING

REST – REPRESENTATIONAL STATE TRANSFER

UI – USER INTERFACE

CSS – CASCADING STYLE SHEETS

HTML – HYPERTEXT MARKUP LANGUAGE

CDN – CONTENT DELIVERY NETWORK

LTS – LONG-TERM SUPPORT

CORS – CROSS-ORIGIN RESOURCE SHARING

HTTP – HYPERTEXT TRANSFER PROTOCOL

MPA – MULTI-PAGE APPLICATION

PPM – PARTS PER MILLION

JSON – JAVASCRIPT OBJECT NOTATION

ID – IDENTIFIER

IOT – INTERNET OF THINGS

MQTT – MESSAGE QUEUING TELEMETRY TRANSPORT

PWA – PROGRESSIVE WEB APPLICATION

Figure No.	Figure Title	Page
Figure 1	IoT Sensor Device Placement and Internal Sensor Components	23
Figure 2	Verification of Column Name Consistency Between Office A and Office B	24
Figure 3	Verification of Data Type Consistency Across Merged Datasets	25
Figure 4	Final Merged Dataset Shape After Integration ($144,000 \times 12$)	25
Figure 5	Sample of Raw Data Before Feature Selection	26
Figure 6	Selected Relevant Features After Data Cleaning	27
Figure 7	Dataset After Converting Month to Numerical Representation	28
Figure 8	Verification of Missing Values in Selected Features	28
Figure 9	Outlier Counts per Feature Using the IQR Method	29
Figure 10	Boxplot Visualization of Relative Humidity (RH) for Outlier Detection	31
Figure 11	Scatter Plot of CO₂ vs CO₂ × Occupancy Interaction Feature	34
Figure 12	Boxplot of CO₂ Distribution by Occupancy State	34
Figure 13	Scatter Plot of CO₂ vs CO₂ × Window Interaction Feature	35
Figure 14	Boxplot of CO₂ Distribution by Window State (Open vs Closed)	35
Figure 15	Distribution of Window × Occupancy Combined Feature	35
Figure 16	Scatter Plot of Indoor Temperature vs Temperature × Relative Humidity	36
Figure 17	Histogram of Temperature × Relative Humidity Feature Distribution	36
Figure 18	Final Dataset After Feature Engineering	36
Figure 19	Distribution of Air_Quality_Label Classes	38

Figure No.	Figure Title	Page
Figure 20	Validation Check — Unlabeled Rows Count (0)	38
Figure 21	Air Quality Label Distribution (Count-Based)	38
Figure 22	Air Quality Label Distribution (Percentage Bar Chart)	39
Figure 23	Air Quality Label Distribution (Percentage-Based Pie Chart)	39
Figure 24	Indoor Air Quality (IAQ) Pipeline	40
Figure 25	System Architecture	44
Figure 26	Data Pipeline Used in The Project	46
Figure 27	Neural Network Architecture	50

CHAPTER1 : INTRODUCTION

This chapter provides a comprehensive overview of the proposed project, highlighting its importance, the motivation behind implementing it, and the main problem it seeks to address. It also discusses the project's objectives, expected outputs, and timeline, in addition to explaining the structure of the report. The purpose of this chapter is to establish a general framework that helps the reader understand the context of the project before examining the technical details presented in the subsequent chapters.

1.1 Overview

The world is witnessing increasing interest in indoor air quality, especially as individuals spend more time inside enclosed environments such as homes, offices, and classrooms. People with asthma or claustrophobia are particularly sensitive to poor indoor air quality, as they may experience breathing difficulties or anxiety attacks without having any early indicator that the environment is unsuitable.

This project focuses on developing an artificial intelligence system capable of evaluating indoor air quality using various environmental data (temperature, humidity, CO₂ level, number of occupants, and room size). The system provides an instant air quality rating, along with smart recommendations aimed at improving the indoor environment.

1.2 Project Motivation

The main motivation behind this project is the absence of a practical and fast method for evaluating indoor air quality before or during exposure, especially for individuals who are at higher health or psychological risk such as asthma patients or those with claustrophobia. Currently, people rely on personal perception or subjective experience to estimate the suitability of an indoor space, which is inaccurate and may lead to worsening symptoms.

Modern artificial intelligence technologies allow the analysis of multisource data to generate accurate assessments and predict air quality before actual symptoms appear. This enables the development of a system that supports a sensitive segment of society and enhances safety within indoor environments.

1.3 Problem Statement

People with asthma or claustrophobia struggle to determine whether the environment around them is safe or suitable before experiencing symptoms. There is no smart and quick method to identify indoor air quality, and the factors affecting air quality are multiple and constantly changing, such as humidity, CO₂ concentration, temperature, and the number of people in the room.

Therefore, the core problem can be described as follows:

There is no data-driven system capable of evaluating indoor air quality in real time and providing suitable recommendations for individuals at higher risk.

1.4 Project Aim and Objectives

Aim

To develop an artificial intelligence system capable of evaluating indoor air quality and alerting individuals who are vulnerable to asthma or claustrophobia, while providing intelligent recommendations to improve the indoor environment.

Objectives

1. Collect environmental data including temperature, humidity, CO₂ level, number of occupants, and room size.
2. Process and analyze the data to identify the key factors affecting air quality.
3. Build an AI model capable of classifying air quality (Excellent / Fair / Poor).
4. Integrate a real sensor (such as humidity or temperature) to create a practical, usable system.
5. Generate automated smart recommendations such as “Open the window” or “Reduce the number of occupants.”
6. Evaluate the model using unseen test data to verify its generalization capability.
7. Develop an initial prototype that can later be expanded into a mobile application or a physical device.

Answers to the required questions:

Q1. What is the goal the project wants to achieve?

The goal is to build an AI system capable of evaluating indoor air quality in real time and warning or guiding individuals who are at higher health or psychological risk.

Q2 . How will the project achieve this goal?

By collecting environmental data, training an artificial intelligence model to classify air quality, linking the model to real sensor readings for real-time assessment, and providing practical recommendations to improve air quality.

1.5 Project Limitations

Several limitations may affect the performance of the project, including:

1. Data limitations: Real-world indoor air quality data may be limited, so simulated data or publicly available datasets will be used for training.
2. Number of sensors used: The system currently relies on only one physical sensor (e.g., humidity or temperature), which may reduce accuracy compared to commercial multi-sensor systems.
3. Environmental variability: Air quality changes rapidly, so the system may require continuous updates or additional data to maintain accuracy.

1.6 Project Expected Output

The project is expected to produce the following outputs:

1. A trained artificial intelligence model for classifying air quality.
2. Real-time air quality assessment using the sensor's input.
3. Smart recommendations for improving indoor air quality.
4. Data analysis reports showing the most influential factors.
5. An initial prototype that can later be expanded.
6. A simple dashboard or interface displaying the results.

1.7 Project Schedule

Phase	Description	Status
Phase 1	Data Collection and Dataset Selection	Completed
Phase 2	Data Understanding and Exploration	Completed
Phase 3	Data Preprocessing and Cleaning	Completed
Phase 4	Data Integration and Transformation	Completed
Phase 5	Feature Engineering and Labeling	Completed
Phase 6	Ethical Considerations and Data Privacy Assessment	Completed
Future Phase	Model Development and Evaluation	Planned

1.8 Report Organization

Chapter 1: Introduction, motivation, problem description, objectives, limitations, and expected outputs.

Chapter 2: Literature Review

Chapter 3: Requirement Engineering and Analysis

Chapter 4: Model Development and Architectural Design

Chapter 5: Model/System Evaluation and Testing Plan

Chapter 6: Model Deployment and Integration

Chapter 7: Conclusion and future work

CHAPTER 2: LITERATURE REVIEW

We explore the existing research on indoor and outdoor air quality prediction using artificial intelligence by reviewing relevant studies. We summarize the key findings and methodologies applied in previous works, including machine learning and deep learning approaches, to provide context for our project. By examining the literature, we discuss the accuracy rates and performance metrics achieved in these studies, offering insight into the effectiveness of various models and techniques for air quality forecasting. In addition, we highlight gaps in the current knowledge and demonstrate how our research builds on and contributes to the existing work in this field, particularly in enhancing prediction reliability and supporting proactive indoor environmental management.

1.This study addresses indoor air quality (IAQ) prediction by focusing on CO₂ concentrations in buildings in Shanghai from November 2016 to March 2017. Artificial intelligence models, including Machine Learning (ML) and Deep Learning (DL) techniques, were applied to sensor-collected datasets to forecast CO₂ levels. Specifically, Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks were implemented to capture temporal patterns in the data. Model performance was evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Median Absolute Error, and Coefficient of Determination (R^2). Results showed that LSTM and GRU provided the highest prediction accuracy, highlighting their suitability for time-series forecasting of IAQ. The study demonstrates the importance of selecting models tailored to dataset characteristics, enabling more reliable and efficient indoor air quality management, and supporting intelligent ventilation and occupant safety.

2.Machine Learning (ML) has been used in environmental research to predict indoor air quality, but few studies focused on PM_{2.5}, fine particles that can reach deep into the lungs and cause respiratory and heart problems. Estimating indoor PM_{2.5} is important, especially for the elderly and people with health issues who spend most of their time indoors. This study used ML models including GLM with Lasso, Random Forest (RF), and XGBoost to predict six-hourly indoor PM_{2.5} in six non-smoking homes in Malta and Gozo, using outdoor PM and weather data. PM was

measured continuously with aerosol spectrometers. Models were tested with repeated 10-fold cross-validation and hyperparameter tuning using RMSE. For homes with low indoor PM, GLM worked well but showed some correlation in results, while RF and XGBoost performed very well (IOA 0.92–0.93), with outdoor PM₁ as the most important factor. When including a home with high PM from cooking, RF gave the best performance (RMSE 30.65 $\mu\text{g}/\text{m}^3$, IOA 0.66) and indoor humidity helped predict high PM. The study showed outdoor PM₁ strongly affects indoor PM_{2.5} where indoor sources are low, while cooking can raise indoor PM_{2.5} up to 3.6 times above WHO guidelines, so kitchen ventilation is important to reduce exposure.

3.Indoor exposure to fine particles (PM_{2.5}) poses health risks, prompting the use of low-cost sensors for monitoring, but their accuracy is affected by environmental conditions and instrument drift. This study introduced an automated machine learning (AutoML) calibration framework to improve the reliability of low-cost indoor PM_{2.5} sensors. The multi-stage framework linked low-cost field sensors with drift-correction reference sensors and a high-grade reference instrument, applying separate calibration models for low (clean air) and high (pollution events) concentrations. The system was tested in a controlled indoor chamber with two sensor models exposed to various indoor pollution sources under natural ambient conditions. AutoML calibration significantly enhanced sensor performance, achieving strong correlation with reference measurements ($R^2 > 0.90$) and substantially reducing error metrics, with RMSE and MAE roughly halved compared to uncalibrated data, while bias was minimized. This study demonstrates that AutoML calibration can convert low-cost sensors into reliable tools for indoor air quality monitoring, supporting more accurate exposure assessment and informing public health interventions.

4.Indoor exposure to bioaerosols, especially in crowded and poorly ventilated spaces, poses significant public health risks, yet real-time monitoring and near-future prediction of airborne biological particles remains challenging. This study developed artificial intelligence (AI) models using data from indoor air quality sensors and ultraviolet light-induced fluorescence observations to estimate concentrations of bioaerosols (bacteria-, fungi-, and pollen-like particles) and particulate matter (PM_{2.5} and PM₁₀) in real-time and up to 60 minutes ahead. Seven AI models were trained and evaluated using data from an occupied commercial office and a shopping mall. The Long Short-Term Memory (LSTM) model achieved the highest prediction accuracy, approximately 60–80% for bioaerosols and around 90% for PM on testing and time series datasets. This study demonstrates that AI-based prediction can provide building operators with near real-time forecasts of indoor bioaerosol and PM concentrations, enabling better management of indoor environmental quality.

5. Monitoring indoor CO₂ levels is essential for health, as elevated CO₂ can cause headaches, fatigue, and impaired cognitive function. This study developed an adaptive IoT-based forecasting system for homes that uses low-cost wireless environmental sensors and a Smartex wearable t-shirt to monitor CO₂, temperature, humidity, and occupant physical activity in real time. The model employs a Generative Adversarial Network (GAN) with LSTM networks as generator and discriminator to predict near-future CO₂ values. Only five days of real data were used for training, while GAN-generated data simulated an additional five days, effectively creating a 10-day dataset. The system was tested in a residential room with one occupant performing smart work and exercise. Results showed high accuracy, with a Root Mean Square Error (RMSE) of approximately 8 ppm, and inclusion of activity data significantly improved predictions (RMSE decreased from 80.22 ppm to 7.78 ppm). This approach allows accurate, multi-step CO₂ forecasting with limited data, supporting proactive ventilation control, energy efficiency, and occupant comfort in real-world indoor environments.

6. This study presents a compact smart air purifier integrating a dual-stage filtration system (Intense Field Dielectric (IFD) for PM removal and zeolite-based filter for VOC adsorption) with high-precision sensors and machine learning (ML)-based predictive control. Computational Fluid Dynamics (CFD) simulations optimized airflow and filter design. The purifier operates automatically based on real-time air quality (AQI), achieving up to 30% energy savings. Experimental results showed a 76% reduction in PM_{2.5} and 65% reduction in VOCs within 30 minutes. Eleven ML regression models were tested for AQI prediction, with Lasso Regression performing best ($R^2 = 0.9179$, RMSE = 5.14, MAE = 4.13). Feature analysis identified PM_{2.5}, PM₁₀, noise, and humidity as key predictors. The system demonstrates how combining advanced filtration with ML forecasting enables anticipatory, energy-efficient indoor air purification suitable for homes, offices, and other enclosed environments.

7. The study assessed air quality in Kuwait using data from 12 monitoring stations (Jan 2019–Nov 2021), including PM_{2.5}, PM₁₀, gases, and meteorological parameters. PM_{2.5} often exceeded “Good” and “Moderate” levels, especially in industrial and high-traffic areas, with “Unhealthy” peaks during summer and dust storms. Severe pollution correlated with temperatures of 15–45°C, humidity <50%, wind speeds <12 m/s, and wind directions 240–340°, showing that dry, stagnant air and winds from industrial zones worsen pollution. Seasonal peaks occurred in summer and spring. Eight ML algorithms were tested across five feature sets; Gradient Boosting and AdaBoost performed best ($R^2 \approx 0.999$) when key PM_{2.5}/PM₁₀ and meteorological features were included, while pruning irrelevant features improved efficiency. Tree-based ensemble methods captured complex non-linear relationships, making deep learning unnecessary. Main pollution drivers were

industrial proximity, meteorology, and PM levels. Policy recommendations include reducing gas flaring, adopting renewables, promoting EVs, and importing cleaner electricity. The study demonstrates that combining environmental data, meteorological factors, and targeted ML feature selection enables accurate air quality prediction and informs effective public health and urban planning strategies.

8. Air pollution is a critical public health challenge in megacities, and traditional numerical or single-site ML models face limitations such as high computational costs and low efficiency. To address this, the FuXi-Air model was developed using multimodal data fusion, integrating meteorological forecasts, emission inventories, and pollutant monitoring data guided by air pollution mechanisms. Utilizing an autoregressive prediction framework combined with frame interpolation, FuXi-Air provides accurate 72-hour forecasts for six major pollutants at hourly resolution across multiple monitoring sites within 25–30 seconds. Ablation studies show that meteorological data contribute most to accuracy, but integrating all data sources significantly improves predictions under varying pollution mechanisms. The model demonstrates superior computational efficiency and forecasting precision compared to traditional models, offering a practical example for deploying multimodal AI-based forecasting systems and supporting smart city air pollution risk management.

9. This study addresses the critical problem of air pollution and its impact on public health by proposing an advanced machine learning approach for predicting PM_{2.5} concentrations. The core idea is to improve prediction accuracy by combining multiple machine learning models rather than relying on a single model. Air quality datasets collected from major cities such as Beijing and Istanbul were used, including historical PM_{2.5} measurements and related environmental data. Several widely used machine learning models, namely Multi-Layer Perceptron (MLP), Support Vector Regression (SVR), and Random Forest (RF), were first trained independently to learn different patterns within the data. The predictions generated by these base models were then integrated using a stacking ensemble framework, where a meta-learner combines and optimizes the outputs of all models to produce the final PM_{2.5} prediction. Model performance was evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2). The stacking ensemble achieved the best performance, with MAE = 6.67, RMSE = 8.80, and $R^2 = 0.91$, outperforming all individual models. These results demonstrate that the stacking ensemble approach provides higher accuracy and better generalization capability for PM_{2.5} prediction compared to traditional machine learning methods.

10. This study proposes an occupant-centric framework for assessing Indoor Environmental Quality (IEQ) by integrating environmental measurements with self-reported comfort and health data. Unlike traditional approaches that focus on single comfort indicators, the proposed method adopts a multimodal data fusion strategy using Transformer-based models to capture complex temporal patterns and relationships across diverse data sources. The framework was evaluated in public indoor spaces and demonstrated high classification performance, achieving 97% accuracy for comfort-based IEQ classification and 96% accuracy for health-based classification, outperforming baseline models. The results highlight the effectiveness of Transformer-based multimodal learning in providing a comprehensive and accurate assessment of indoor environmental conditions.

11. This research presents an image-based deep learning approach for predicting air quality, focusing on PM_{2.5} concentration. The proposed model combines a three-dimensional convolutional neural network (3D-CNN) with a gated recurrent unit (GRU) and an attention mechanism. The 3D-CNN extracts spatial and temporal features from multi-dimensional image datasets, while the GRU captures temporal dependencies efficiently. The attention mechanism adjusts the influence of features, improving prediction accuracy and reducing random fluctuations. The model was tested using images from Shanghai, pre-processed to remove noise and normalized, with data split into training and testing sets. Results show that the 3D-CNN-GRU with attention outperforms traditional CNN-LSTM and other baseline models, achieving a mean absolute percentage error (MAPE) of 15.6% and MASE of 21.57 for PM_{2.5}. The model provides accurate multi-horizon forecasts, enabling near real-time air quality monitoring and early warning for pollution. This approach demonstrates the potential of deep learning and image-based data to improve urban air quality prediction.

12. Air pollution is a global problem that threatens environmental sustainability and public health. This study focuses on the long-term prediction of daily Air Quality Index (AQI) in eastern Türkiye from 2016 to 2024, using a dataset that includes four major air pollutants (PM₁₀, SO₂, NO₂, O₃) and five meteorological variables (temperature, precipitation, relative humidity, wind direction, wind speed). Three machine learning models—XGBoost, LightGBM, and Support Vector Machine (SVM)—were implemented to forecast AQI levels. The methodology involved preprocessing the data, splitting it into training and testing sets, and training each model to capture the relationships between environmental predictors and AQI values. Model performance was evaluated using the coefficient of determination (R^2), root mean square error (RMSE), and mean absolute error (MAE). Among the three models, XGBoost achieved the highest prediction accuracy with $R^2 = 0.999$, RMSE = 0.234, and MAE = 0.158, outperforming LightGBM and SVM. These results demonstrate that ensemble-based machine learning approaches like XGBoost can

effectively model AQI fluctuations using environmental predictors, providing valuable insights for air quality forecasting systems, early warning mechanisms, public health protection, and environmental policy-making.

13. This study focuses on predicting the Air Quality Index (AQI) using multiple machine learning approaches, including K-Nearest Neighbors (KNN), XGBoost, and a Neural Network (NN). The dataset includes daily measurements of major air pollutants (PM_{2.5}, PM₁₀, NO₂, SO₂, O₃) along with meteorological variables such as temperature, humidity, wind speed, and wind direction. The methodology involved preprocessing the data, normalizing features, and splitting the dataset into training and testing sets. The KNN model was applied to forecast AQI values by identifying similar historical patterns in the data. Results showed that KNN achieved an accuracy of 85.39%, performing better than XGBoost (72.91%) but slightly lower than the Neural Network model, which achieved 98.17% accuracy. The study demonstrates that KNN is effective for capturing local patterns in environmental data and provides a reliable baseline for air quality prediction, especially in short-term forecasting scenarios.

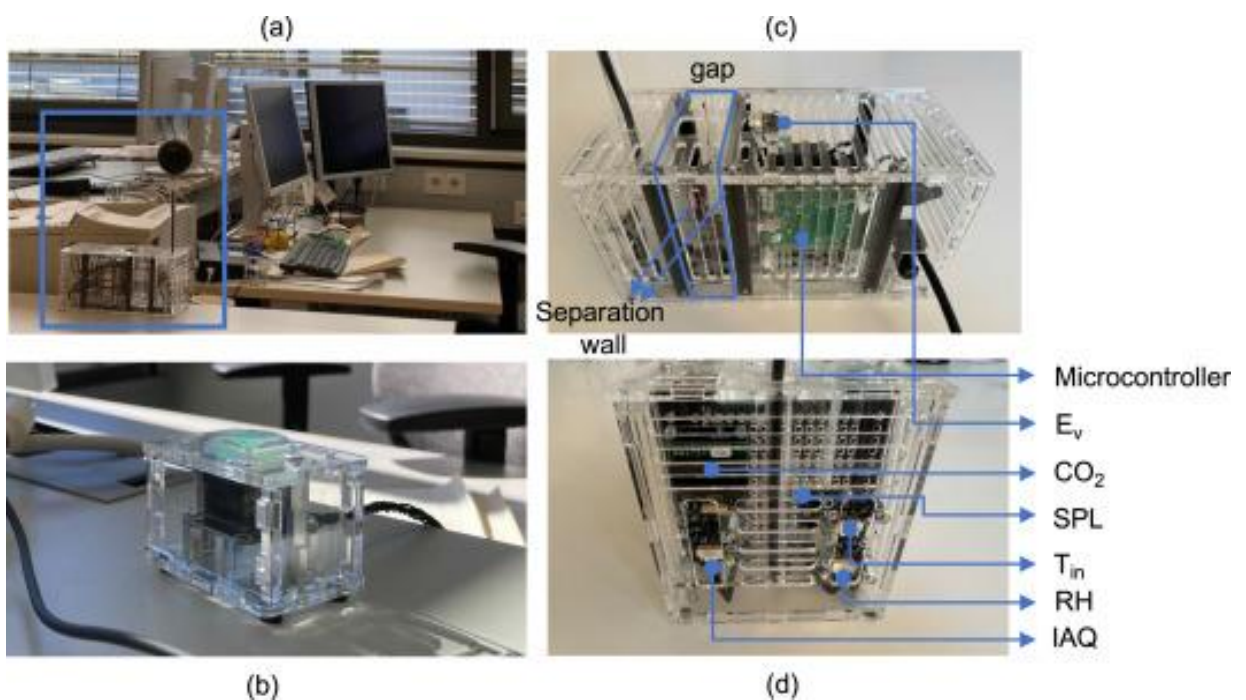
Chapter 3: Requirement Engineering and Analysis

Data Collection Methods and Sources

The data used in this project was obtained from a publicly available dataset titled *“Non-intrusive indoor environmental data in two double-occupied offices”*. The measurements were collected as part of a longitudinal study conducted in two naturally ventilated offices at the Munich University of Applied Sciences in Germany. Each office is equipped with manually operable windows, exterior shading, and a heating system.

An **IoT** sensory device was installed at desk level near the occupants to continuously monitor the indoor environment. The sensors recorded several physical variables, including **air temperature**, **CO₂ concentration**, **indoor air quality (IAQ) index**, **illuminance**, **relative humidity**, **sound pressure level**, and **air speed**, **alongside the window status**. Occupancy ground truth was recorded as a **binary label** (**occupied vs. unoccupied**), while the window state (**open/closed**) was transmitted via an **EnOcean**-based contact sensor.

The IAQ index in this dataset is defined on a scale from 0 to 500, where smaller values indicate better indoor air quality and larger values represent increasingly degraded conditions. Internally, the index is derived from the concentration of



multiple volatile organic compounds, providing a compact indicator of overall indoor air pollution.

Figure1

The raw data is distributed as two CSV files, **office_A.csv** and **office_B.csv**, each corresponding to one of the monitored offices. Both files were downloaded from the Mendeley Data repository and form the starting point for the preprocessing and analysis steps described in the following subsections.

Why We Merged the Two Datasets (office_A and office_B)

Both **office_A** and **office_B** contain the same structure, sensor measurements, and data collection methodology. Therefore, merging the two files into a single unified dataset was an essential step for several reasons:

First, merging increases the overall dataset size, which enhances the model's ability to generalize and improves prediction accuracy.

Second, combining data from two different office environments introduces more variability in environmental conditions and occupant behavior, making the dataset more representative.

Third, working with one consolidated dataset simplifies preprocessing, analysis, and model development.

Figure 2

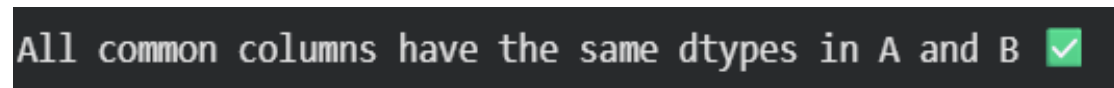
```
Same columns (same order)? True
Columns only in A: []
Columns only in B: []
Same column names (ignoring order)? True
```

Verifying Column Consistency Before Merging

Before merging the two datasets (*office_A* and *office_B*), it was essential to verify that they shared the same column names and structure. Ensuring column consistency prevents alignment issues, incorrect data mapping, and potential loss of information

during the merge operation. Therefore, we checked whether both files contained identical column names, in the same order and with no missing or additional fields. The verification results confirmed that the datasets matched perfectly, allowing us to safely proceed with merging them into a unified dataset.

Figure 3



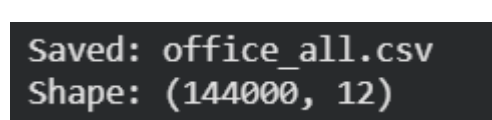
```
All common columns have the same dtypes in A and B ✓
```

Before merging the two datasets (**office_A** and **office_B**), it was crucial to ensure that both files used the same data types for all shared columns. Differences in data types—such as one file storing a column as *float* while the other stores it as *int*—can lead to merge errors, incorrect calculations, or unexpected model behavior later in the workflow.

To avoid these issues, we performed a **data type alignment check** on all common columns across the two datasets. The results showed that **all shared columns have identical data types in both files**, confirming that the datasets are structurally compatible and safe to merge into a unified dataset.

This verification step ensures clean integration and prevents downstream inconsistencies during preprocessing and model development.

Figure 4



```
Saved: office_all.csv  
Shape: (144000, 12)
```

After verifying the column consistency and data types, the two raw files (**office_A.csv** and **office_B.csv**) were concatenated into a single integrated dataset named **office_all.csv**. The resulting dataset contains **144,000 rows and 12 columns**, providing a large number of indoor environmental measurements for subsequent analysis and modelling.

Figure5

Unnamed: 0	DAY	TIME	MONTH	OCCUPANCY	WINDOW	Ev	IAQ	CO2	SPL	RH	Tin	
0	0	1	0:00:00	September	0	0	0.01	72.40	404.25	27.86	53.45	26.18
1	1	1	0:01:00	September	0	0	0.01	73.05	404.07	27.88	53.45	26.18
2	2	1	0:02:00	September	0	0	0.01	74.08	404.07	27.79	53.45	26.18
3	3	1	0:03:00	September	0	0	0.01	72.27	406.42	27.77	53.42	26.18
4	4	1	0:04:00	September	0	0	0.01	73.98	407.35	27.79	53.45	26.17

During preprocessing, several columns were removed because they did not contribute meaningful information or were redundant for the machine learning workflow. These include:

1. Unnamed: 0

An automatically generated index column that does not contain any real sensor information.

It was removed as it has no analytical value.

2. DAY and TIME

These timestamp components do not provide useful predictive information for the indoor air quality task.

Since the project does not involve time-series modeling, keeping them would only introduce unnecessary noise.

3. Ev

This column had a constant value (~0.01) across the entire dataset.

Constant features do not contribute to model learning and are therefore removed.

Figure6

	MONTH	OCCUPANCY	WINDOW	IAQ	CO2	RH	Tin
0	September	0	0	72.40	404.25	53.45	26.18
1	September	0	0	73.05	404.07	53.45	26.18
2	September	0	0	74.08	404.07	53.45	26.18
3	September	0	0	72.27	406.42	53.42	26.18
4	September	0	0	73.98	407.35	53.45	26.17

Data Type Standardization

As part of the preprocessing phase, we examined the data types of all columns to ensure consistency and compatibility with the subsequent analysis steps. During this inspection, we found that all variables were already represented in numeric format except for the MONTH column, which was stored as a text value (e.g., “September”). Since non-numeric values can cause issues during analysis and cannot be directly used in machine learning workflows, the column was transformed into a numerical representation called **MONTH_NUM** reflecting the actual month number.

It is important to note that this transformation **does not constitute an encoding technique** such as label encoding or one-hot encoding. The month variable was not treated as a categorical label; instead, it was converted into a natural numeric form because months inherently follow an ordered temporal structure. Therefore, this step falls under **data standardization**, not encoding.

After this conversion, all columns in the dataset were fully numeric, ensuring that the data is clean, consistent, and ready for the next stages of preprocessing and modeling.

Figure7

	OCCUPANCY	WINDOW	IAQ	CO2	RH	Tin	MONTH_NUM
0	0	0	72.40	404.25	53.45	26.18	9
1	0	0	73.05	404.07	53.45	26.18	9
2	0	0	74.08	404.07	53.45	26.18	9
3	0	0	72.27	406.42	53.42	26.18	9
4	0	0	73.98	407.35	53.45	26.17	9

Figure8

	0
OCCUPANCY	0
WINDOW	0
IAQ	0
CO2	0
RH	0
Tin	0
MONTH_NUM	0
dtype:	int64

Missing Values Analysis

we examined the dataset for missing values. For this purpose, we computed the number of null entries in each column using the `isna().sum()` function. The results

showed that all features (OCCUPANCY, WINDOW, IAQ, CO₂, RH, Tin, and MONTH_NUM) contained **zero missing values**.

Since no null or undefined entries were present in the integrated dataset, no imputation or row removal was required at this stage. This confirms that the data is complete and ready for subsequent preprocessing and analysis steps.

Outlier Detection Using the IQR Method

To ensure the reliability of the minute-level sensor data, we performed outlier detection using the Interquartile Range (IQR) method. This technique was selected because environmental sensor readings (such as CO₂, humidity, temperature, and IAQ) naturally fluctuate throughout the day, and not every extreme value should be treated as an error.

The IQR method is particularly suitable for this type of data for several reasons:

1. **It is robust to extreme values** and does not rely on the mean, making it ideal for skewed or non-normally distributed sensor data.
2. **It avoids misclassifying natural fluctuations as anomalies**, which is essential when working with high-resolution indoor environmental measurements.
3. **It is widely used in scientific literature** for analyzing environmental and IoT sensor datasets due to its simplicity and stability.
4. Importantly, in our workflow, the IQR method was applied **only to identify outliers**, not to remove or modify them, ensuring that the raw environmental patterns remain intact for accurate modeling.

The analysis revealed outliers in a few numerical features (e.g., CO₂, RH, Tin, IAQ), which helped us understand the variability and noise level in the dataset without compromising its integrity.

Figure9

```
Outliers per column (IQR):  
{ 'CO2': 2089, 'RH': 40, 'Tin': 5873, 'IAQ': 17 }
```

IMPORTANT

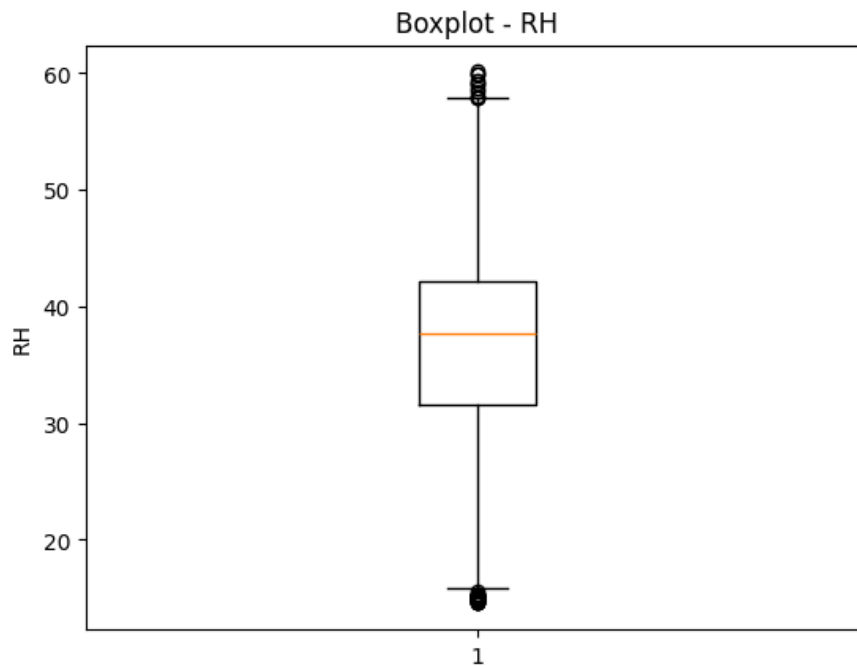
Why Outliers Were Detected but Not Removed

During the preprocessing stage, we applied the Interquartile Range (IQR) method to identify potential outliers in key environmental variables such as CO₂, IAQ, temperature, and relative humidity. However, although outliers were detected, **we intentionally chose not to remove them**. This decision is critical for maintaining the integrity and realism of the dataset.

In indoor environmental monitoring, extreme or unusual readings are not necessarily sensor errors—they often represent **real and meaningful environmental events**, such as sudden increases in occupancy, poor ventilation, window closure, or conditions that may trigger discomfort or health risks for sensitive individuals (e.g., asthma patients or people with claustrophobia). Such rare but important events reflect high-risk scenarios that the predictive model must learn to recognize. Removing them would eliminate essential patterns related to unsafe or abnormal environmental conditions and would lead to a model that performs well only under normal circumstances but fails when detecting critical deviations.

For these reasons, IQR-based outlier detection was used **solely for analysis and understanding the behavior of the data**, not for filtering or modifying it. All values were retained to ensure that the dataset remains realistic, representative, and capable of supporting a robust machine learning model that can generalize to both typical and extreme environmental situations.

Figure10



Boxplot Visualization for Outlier Identification

To visually inspect the presence of outliers in the environmental features, we used boxplots. The figure above shows an example boxplot for the **relative humidity (RH)** feature. In a boxplot, the central box represents the interquartile range (IQR), which spans from the 25th percentile (Q1) to the 75th percentile (Q3). The horizontal line inside the box indicates the median value of the distribution.

The “whiskers” extend from the box to the lowest and highest values that fall within an acceptable range, typically defined as $1.5 \times \text{IQR}$ below Q1 and above Q3. Any data points plotted as isolated markers beyond the whiskers are considered **potential outliers**.

In the RH boxplot, the scattered points above the upper whisker and below the lower whisker represent readings that lie outside the normal humidity range observed in the office environment. This graphical technique complements the IQR-based

numerical analysis by providing an intuitive view of how extreme values are distributed relative to the bulk of the data.

```
np.int64(6)
```

When checking for duplicate rows using **df.duplicated().sum()**, a small number of records appeared as duplicates based on the sensor values only (e.g., CO₂, RH, Tin, IAQ). However, closer inspection showed that these rows did **not** share the same timestamp. In other words, the environmental measurements were identical or very similar, but they were recorded at **different time points**.

Such repetitions are expected in indoor environmental data, where conditions can remain stable over several minutes. Therefore, these rows do not represent true duplicates of the same event, but rather valid observations of similar conditions occurring at different times. For this reason, we chose **not to remove** them, and all rows were retained to preserve the full temporal structure of the dataset.

Noise / Validity Checks for Sensor Data

To ensure that the dataset does not contain unrealistic or noisy measurements, we applied a series of rule-based validity checks for each feature, based on the sensor descriptions in the paper:

- **OCCUPANCY (0 or 1)**

The OCCUPANCY column represents whether the office is occupied or not. We verified that all values are either 0 (no occupants) or 1 (at least one occupant). Any value not in {0, 1} would be flagged as invalid. The check returned 0 invalid values, confirming consistent binary encoding.

- **WINDOW (0 or 1)**

The WINDOW feature encodes the window state. Similar to occupancy, we

enforced a binary domain with 0 = closed and 1 = open. Values outside this set are considered noisy or mis-recorded. No violations were detected for this column.

- **Relative Humidity RH ([0, 100])**

For RH, which represents relative humidity in percent, we constrained the readings to the physically meaningful range [0, 100]. Any value below 0% or above 100% was treated as implausible and thus noise. The validation showed that all humidity values lie within this range (0 invalid readings).

- **CO₂ concentration (> 0)**

The CO₂ column stores the indoor carbon dioxide concentration in ppm. Since CO₂ cannot be zero or negative in a real environment, we required all values to be strictly greater than 0. The check confirmed that there are no non-positive CO₂ values in the dataset.

- **Indoor Air Quality Index IAQ ([0, 500])**

The IAQ feature is an indoor air quality index defined in the original study, with a theoretical range from 0 (good air quality) to 500 (very poor air quality). We therefore verified that all IAQ values fall within the interval [0, 500]. Values outside this range would indicate sensor or processing errors. The noise check returned 0 out-of-range IAQ values, suggesting that the recorded index values are consistent with the expected specification.

Overall, all these checks reported zero invalid entries, which indicates that the dataset is free from obvious noise or corrupted measurements according to our domain-based validation rules.

Feature Engineering and Selection

CO₂–Occupancy Interaction Feature (CO₂_x_OCCUPANCY)

Figure11

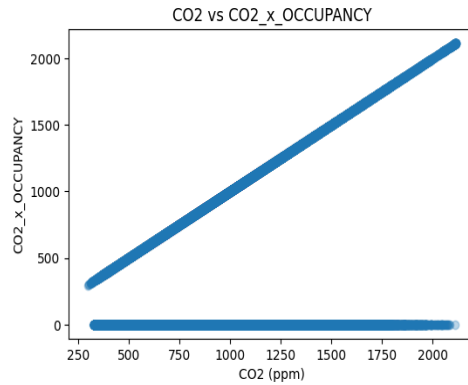
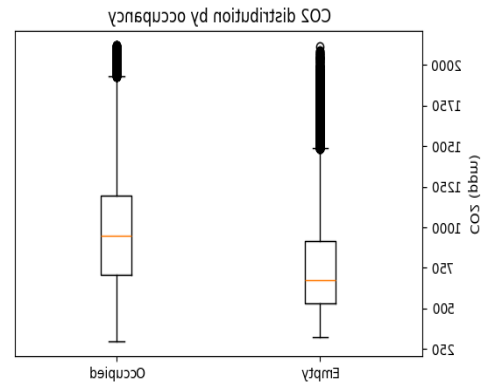


Figure12



To capture how indoor CO₂ levels relate specifically to human presence, we created an interaction feature by multiplying the CO₂ concentration with the binary occupancy indicator (0/1). When the office is empty, this feature is zero, but when occupants are present it equals the measured CO₂ value. This transformation helps the model distinguish between “background” CO₂ fluctuations and CO₂ that is directly associated with human activity, which is particularly relevant for people with asthma or sensitivity to poorly ventilated spaces. By explicitly encoding this interaction, the model can better learn patterns where high CO₂ levels *while occupied* indicate degraded air quality and potential discomfort, rather than treating all CO₂ readings as equally important.

CO₂–Window Interaction Feature (CO₂_x_WINDOW)

Figure13

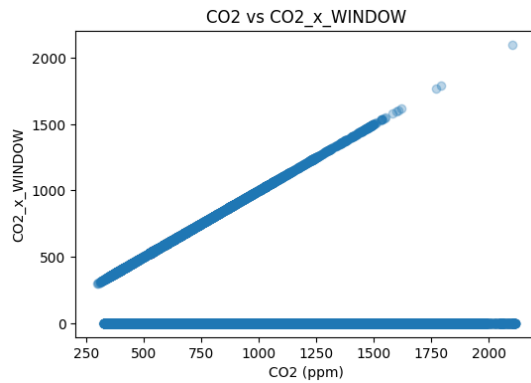
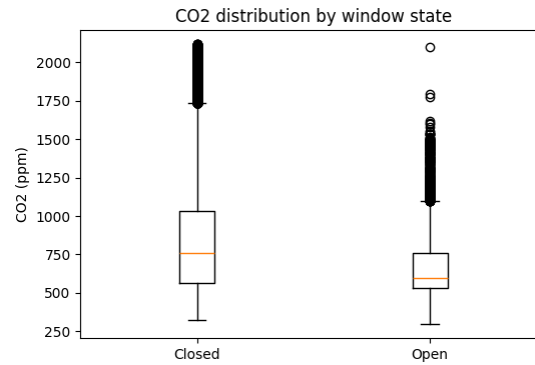


Figure14



We also engineered an interaction feature between CO₂ concentration and the binary window state. The **CO₂_x_WINDOW** feature is equal to the measured CO₂ value when the window is open and zero otherwise. This feature allows the model to explicitly capture how effective natural ventilation is in reducing CO₂ levels. For example, high CO₂ values when the window is open may indicate insufficient airflow or overcrowding, while high CO₂ with the window closed is expected but still relevant for risk assessment. Encoding this interaction helps the model distinguish between “ventilated” and “non-ventilated” high-CO₂ situations, which is important for generating meaningful recommendations such as opening a window or reducing occupancy.

Combined Window–Occupancy Indicator (WINDOW_x_OCCUPANCY)

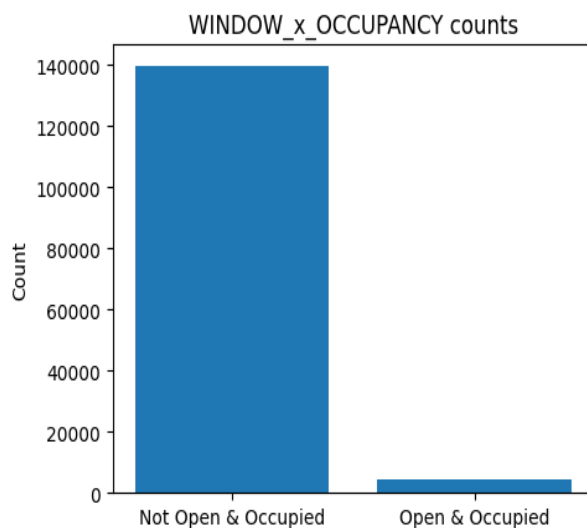
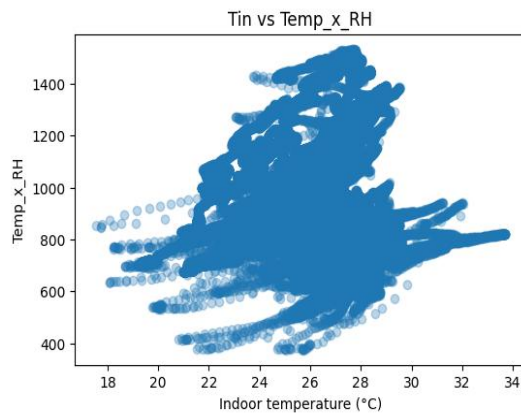
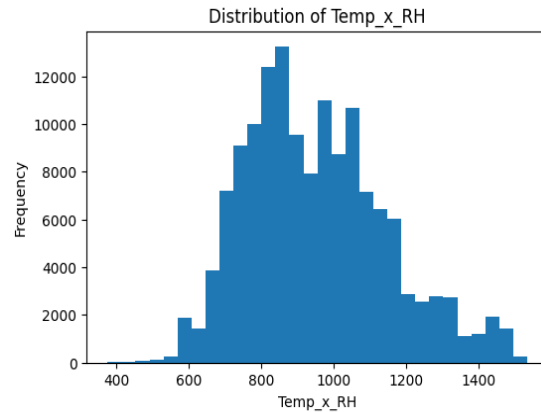


Figure15

To explicitly represent situations where occupants are present *and* the window is open, we created a binary interaction feature by multiplying the window state (0/1) with the occupancy indicator (0/1). The resulting **WINDOW_x_OCCUPANCY** feature equals 1 only when people are in the office and the window is open; in all other cases it is 0. This feature is useful because it directly captures whether ventilation is being used while the room is occupied, which is critical for maintaining healthy air quality. It also simplifies the model’s reasoning: instead of learning complex patterns over two separate binary variables, the model can rely on a single, semantically meaningful indicator of “ventilated-and-occupied” conditions.

Figure16**Figure17**

Human comfort in indoor environments is influenced not only by temperature or humidity alone, but by their combination. To capture this relationship, we engineered a composite feature Temp_x_RH by multiplying indoor temperature (Tin) with relative humidity (RH). This simple proxy provides the model with a continuous measure that increases when both temperature and humidity are high—conditions that are often perceived as stuffy or uncomfortable and may trigger symptoms for individuals with asthma or claustrophobia. While it is not a full psychrometric comfort index, this engineered feature helps the model better detect “uncomfortable air” scenarios compared to using temperature and humidity as separate, independent inputs.

Figure18

	OCCUPANCY	WINDOW	IAQ	CO2	RH	Tin	MONTH_NUM	CO2_x_OCCUPANCY	CO2_x_WINDOW	WINDOW_x_OCCUPAN	WINDOW_x_OCCUPANCY	Temp_x_RH
0	0	0	72.40	404.25	53.45	26.18	9	0.0	0.0	0	0	1399.3210
1	0	0	73.05	404.07	53.45	26.18	9	0.0	0.0	0	0	1399.3210
2	0	0	74.08	404.07	53.45	26.18	9	0.0	0.0	0	0	1399.3210
3	0	0	72.27	406.42	53.42	26.18	9	0.0	0.0	0	0	1398.5356
4	0	0	73.98	407.35	53.45	26.17	9	0.0	0.0	0	0	1398.7865

Target Variable Definition (Labeling) Using the Indoor Air Quality (IAQ) Index

Important

The **Indoor Air Quality (IAQ)** index was used in this project as the reference indicator (ground truth) for defining air quality categories. IAQ is not an arbitrary or subjective metric; rather, it is a **standardized composite index** computed using the **Bosch BSEC (Bosch Sensortec Environmental Cluster) algorithm**, which is widely applied with Bosch gas sensors such as the **BME680/BME688**. This algorithm converts raw gas sensor readings related to volatile organic compounds (VOCs), together with environmental compensation factors, into a normalized IAQ score ranging from **0 to 500**, where lower values indicate better air quality.

According to the official Bosch BSEC documentation, the IAQ scale is associated with qualitative air quality descriptions. Typical classifications define ranges such as *good*, *average*, *poor*, and *very poor* air quality based on specific IAQ intervals. These categories are commonly used in industrial applications, smart building systems, and indoor environmental monitoring studies.

Based on this **established and documented classification**, the IAQ values in this project were mapped into three consolidated categories to simplify interpretation while preserving their scientific meaning. Specifically, IAQ values from **0 to 100** were labeled as **Good**, values from **101 to 200** as **Moderate**, and values greater than **200** as **Poor**. This consolidation merges closely related subcategories from the original scale into broader classes that are easier to interpret and communicate, without deviating from the underlying standard.

Importantly, using IAQ as the sole basis for labeling helps avoid **data leakage**, which may occur if individual environmental variables (such as CO₂ concentration, humidity, or temperature) are directly or indirectly used to define the target classes. By relying on IAQ as an independent reference index and keeping other environmental measurements separate from the labeling process, the categorization remains methodologically sound and unbiased.

This labeling strategy ensures that the defined air quality classes are **standard-compliant, scientifically justified, and interpretable**, providing a reliable foundation for subsequent analysis and evaluation of indoor air conditions.

Figure19

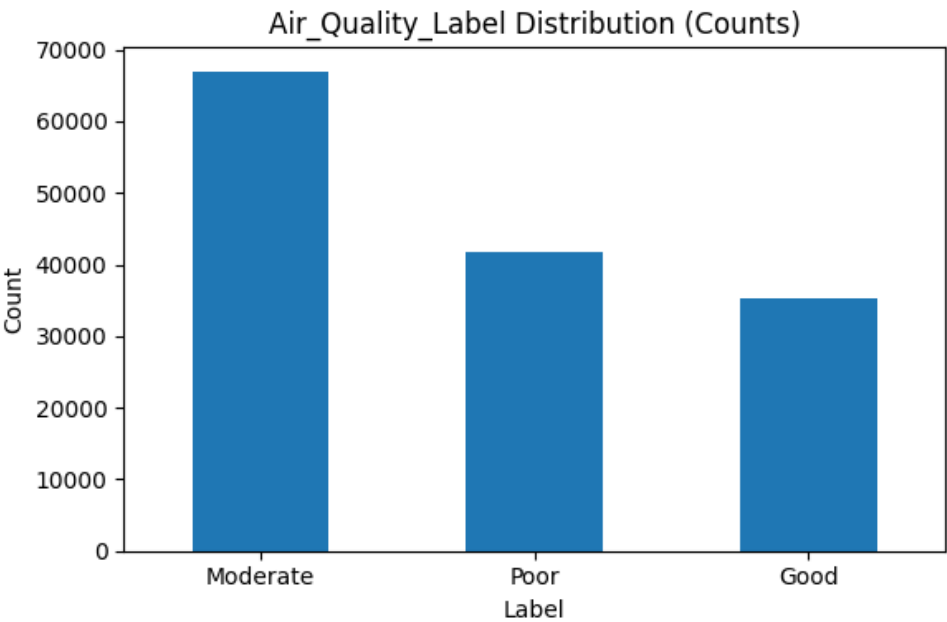
Air_Quality_Label		count
Moderate		66942
Poor		41740
Good		35318

dtype: int64

Figure20

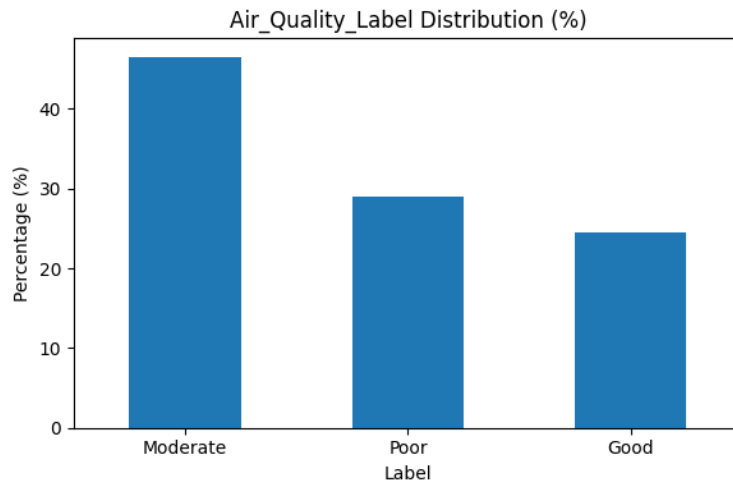
np.int64(0)

Figure21



This figure illustrates the absolute number of samples in each air quality class (Good, Moderate, Poor) after labeling based on the IAQ index. It provides an overview of the class distribution in the dataset and highlights the dominance of the Moderate air quality category.

Figure22



This bar chart presents the percentage distribution of air quality labels. Compared to the count-based representation, this figure emphasizes the relative weight of each class and supports decisions related to model evaluation and potential class imbalance handling.

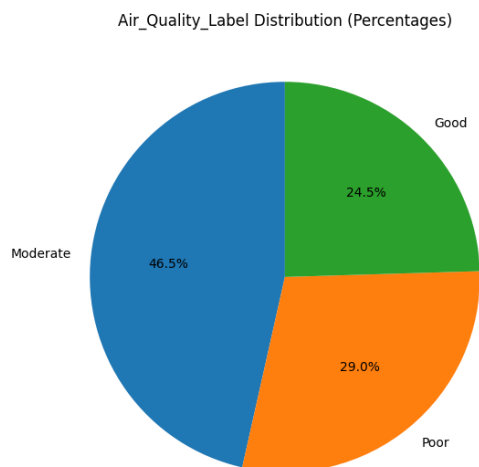
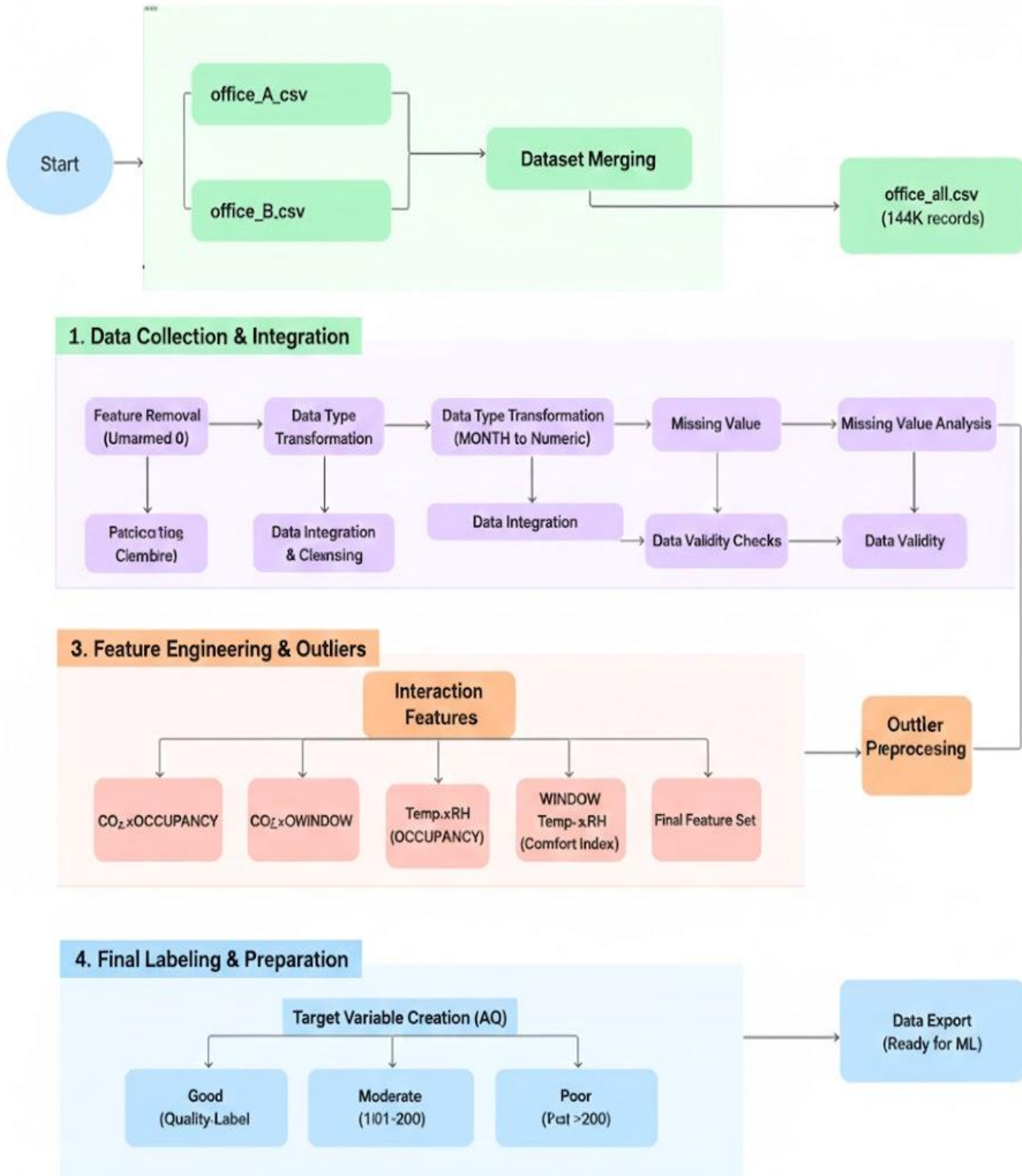


Figure23

This pie chart shows the relative percentage of each air quality label in the dataset. The visualization helps in understanding the proportional representation of Good, Moderate, and Poor air quality conditions, which is important for assessing class balance

Figure24

Indoor Air Quality (AQ) Prediction & Classification Pipeline



3.3 Ethical Considerations and Data Privacy

At this stage, the project focuses exclusively on data understanding and preprocessing, without progressing to final prediction, deployment, or real-world application. Throughout this phase, ethical considerations and data privacy principles were carefully addressed to ensure responsible and transparent handling of the data.

The dataset used in this project consists solely of non-intrusive environmental measurements, including indoor temperature, relative humidity, carbon dioxide concentration, window status, and occupancy indicators. No personally identifiable information (PII) or sensitive personal or health-related data were collected, stored, or processed at any point during this work.

The data were obtained from a publicly available research dataset intended for academic use. Issues related to informed consent were addressed at the data collection stage by the original data providers, while the current project is limited to secondary data analysis. All data were analyzed in an anonymized and aggregated form, ensuring that individual occupants cannot be identified directly or indirectly.

It is important to emphasize that this project does not aim to provide medical diagnosis or clinical recommendations. The analysis is strictly limited to assessing general indoor air quality conditions based on environmental indicators. Any potential future use of the system would be advisory and informational in nature and should not be considered a substitute for professional medical judgment or healthcare services.

Finally, the scope of this project aligns with general data protection principles and regulations such as the General Data Protection Regulation (GDPR) and the Health Insurance Portability and Accountability Act (HIPAA). As no personal or health-related data are involved, the project maintains ethical compliance, data integrity, and privacy protection throughout the preprocessing stage.

CHAPTER 4: Model Development and Architectural Design

4.1 System Architecture Overview

The proposed indoor air quality prediction system follows a modular architecture that supports the full machine learning workflow, from raw environmental data collection to intelligent air quality classification and recommendations. This modular design ensures maintainability, scalability, and flexibility, allowing future integration of new models or additional environmental variables.

The system processes key indoor environmental measurements such as CO₂ concentration, temperature, relative humidity, occupancy, and window state. These variables are critical indicators of indoor air quality and human comfort. By analyzing their relationships, the system classifies indoor environments into predefined air quality categories.

The architecture consists of sequential components:

Data Collection & Integration: Sensor readings from two office environments are merged into a unified dataset, increasing diversity and representativeness for model training.

Data Preprocessing & Validation: Raw data undergo cleaning, type verification, missing value checks, and outlier detection (IQR method). Extreme readings are preserved to capture real-world air quality events.

Feature Engineering: Additional features (e.g., CO₂ × Occupancy, Temperature × Humidity) are generated to capture interactions between environmental conditions and human activity.

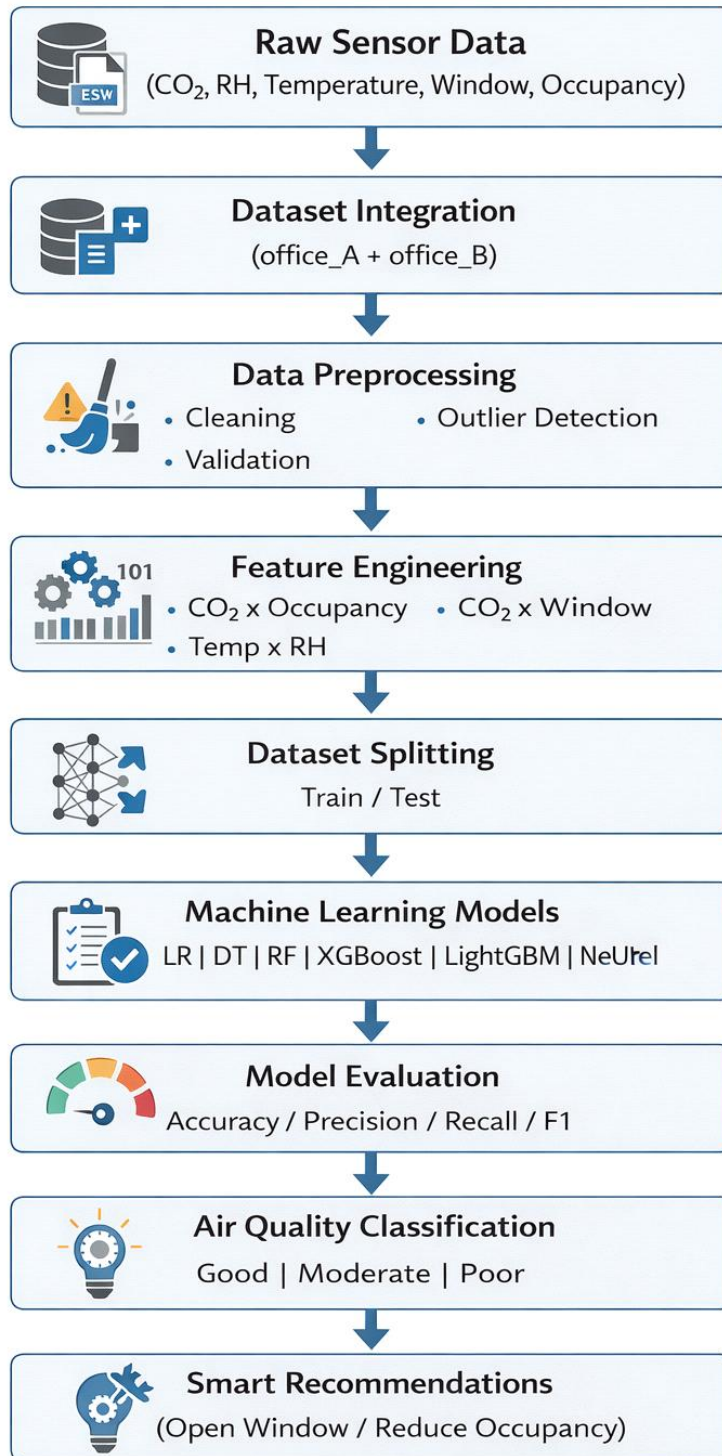
Model Training: Multiple algorithms (Logistic Regression, Decision Tree, Random Forest, XGBoost, LightGBM, Neural Networks) are trained and compared to identify the best performing model.

Model Evaluation & Validation: Models are tested on a separate dataset using metrics such as accuracy, precision, recall, and F1-score to ensure reliable performance.

Prediction & Recommendations: The system predicts air quality (Good, Moderate, Poor) and provides actionable suggestions, such as improving ventilation or adjusting occupancy, to enhance indoor conditions.

This integrated framework combines sensing, preprocessing, modeling, and decision support. Its modular nature allows easy future extension with additional sensors, real-time data, or advanced AI models. **Figure 4.1** illustrates the overall system architecture and interactions between its main components.

System Architecture



4.2 Data Pipeline Design

The data pipeline defines the sequence of processes that convert raw data into a structured format suitable for machine learning models. The pipeline ensures that data is clean, consistent, and appropriately organized for training and evaluation.

The steps in the data pipeline are as follows:

Data Loading:

The raw dataset is imported into the system from its source (e.g., CSV files, databases, or APIs).

Data Cleaning:

Missing values, duplicate entries, and inconsistencies are handled to improve data quality. This step may include imputing missing values, correcting data types, and removing irrelevant columns.

Feature Preparation:

The dataset is transformed into features suitable for machine learning. This may involve:

- Encoding categorical variables

- Scaling numerical variables

- Creating derived features from existing data

Data Splitting:

The processed dataset is divided into training and testing sets.

Training set: Used to train the machine learning models.

Testing set: Used to evaluate the model's performance on unseen data.

Model Learning and Evaluation:

The prepared training data is used to train models (e.g., Random Forest), while the testing data provides an unbiased estimate of model performance.

Figure 4.2 visually represents this pipeline, showing the sequential flow from raw data to model evaluation.

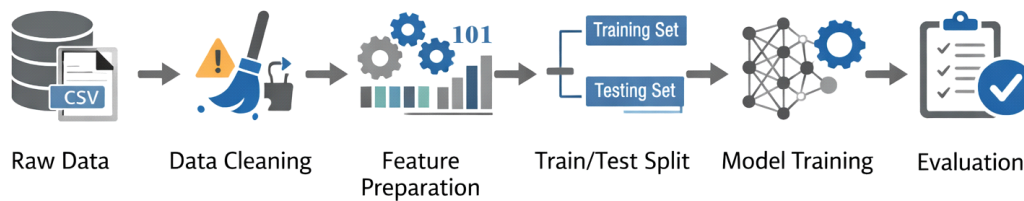


Figure 4.2: *Data Pipeline Used in the Project*

4.3 Model Architecture and Design

In this study, six different machine learning models were implemented and evaluated to predict indoor air quality levels based on environmental sensor data. The selected models include Logistic Regression, Decision Tree, Random Forest, XGBoost, LightGBM, and a Neural Network. These models represent different categories of machine learning algorithms, allowing a comprehensive comparison between traditional statistical methods, tree-based models, ensemble learning techniques, and deep learning approaches.

The main objective of using multiple models was to determine which algorithm can most effectively capture the relationships between environmental variables such as CO₂ concentration, temperature, relative humidity, window status, and occupancy level, and the corresponding air quality classification. Each model processes the same

set of input features but applies different learning mechanisms to identify patterns in the data.

Logistic Regression was used as a baseline model because it is a simple yet powerful statistical classification algorithm that models the probability of a class using a logistic function. It provides interpretable coefficients that indicate how each feature influences the prediction.

Decision Tree was included as a non-linear model capable of capturing complex relationships between variables. Decision trees split the dataset into smaller subsets based on feature thresholds, forming a hierarchical structure that makes predictions through a sequence of decision rules.

Random Forest, an ensemble learning algorithm, was applied to improve prediction performance and reduce overfitting. It builds multiple decision trees using random subsets of the data and features, then combines their predictions through a majority voting mechanism. This approach increases model robustness and accuracy.

Two advanced gradient boosting algorithms, XGBoost and LightGBM, were also implemented. These models build trees sequentially, where each new tree attempts to correct the errors made by the previous ones. Gradient boosting techniques are known for their high predictive performance and efficiency when dealing with structured tabular datasets.

Finally, a Neural Network model was developed to explore the potential of deep learning in capturing complex nonlinear relationships between environmental factors and indoor air quality conditions. The neural network consists of multiple layers of interconnected neurons that learn patterns through backpropagation and gradient-based optimization.

All models were trained using the prepared training dataset and evaluated using the testing dataset. Several performance metrics were used to assess model effectiveness, including Accuracy, Precision, Recall, and F1-Score. These metrics provide a comprehensive understanding of how well each model performs in correctly classifying air quality levels.

The comparative evaluation of these models enables the identification of the most suitable algorithm for predicting indoor air quality. In this study, the results indicated that the Random Forest model achieved the best overall performance, demonstrating strong predictive capability and stability compared to the other algorithms

Table 4.3.1 Decision Tree Hyperparameters

Parameter	Value
max_depth	12
min_samples_split	10
min_samples_leaf	5
criterion	gini

Table 4.3.2 Random Forest Hyper parameters

Parameter	Value
n_estimators	200
max_depth	None
min_samples_split	2
min_samples_leaf	1

Table 4.3.3 XGBoost Hyper parameters

Parameter	Value
n_estimators	200
learning_rate	0.1
max_depth	8
subsample	0.8
colsample_bytree	1.0

Table 4.3.4 LightGBM Hyper parameters

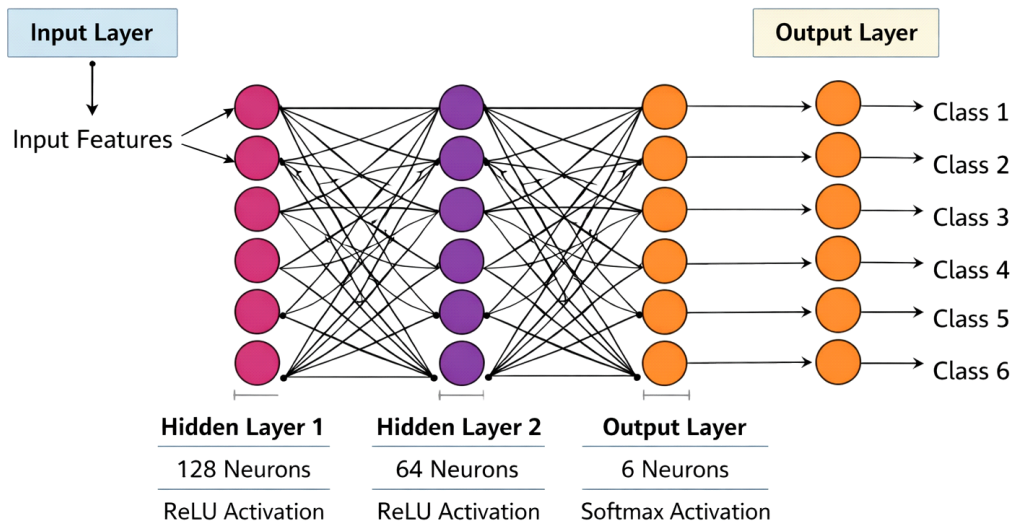
Parameter	Value
n_estimators	500
learning_rate	0.08
num_leaves	90

Table 4.3.5 Neural Network Architecture

Layer	Neurons	Activation
Input	Features	-
Hidden1	128	ReLU
Hidden2	64	ReLU
Output	6	Softmax

Figure 4.3 shows the architecture of the neural network model.

Figure 4.3: Neural Network Architecture



4.4 Training Process

The training process is a critical stage in developing machine learning models, where algorithms learn patterns from historical data in order to make accurate predictions. In this study, several supervised machine learning models were trained using environmental sensor data collected from indoor office environments.

4.4.1 Data Splitting

Before training the models, the dataset was divided into training and testing sets. A stratified split approach was used to maintain the same distribution of air quality classes across both subsets.

The dataset was divided using an 80/20 ratio, where:

80% of the data was used for model training.

20% of the data was reserved for model testing and performance evaluation.

The training dataset allows the models to learn the relationships between input features and the target variable, while the testing dataset evaluates how well the trained model generalizes to unseen data.

In this project, the dataset files were pre-split into training and testing sets, which ensures consistency during experimentation and avoids data leakage between the training and evaluation phases.

Table 4.4.1 Data Split Summary

Dataset	Samples	Ratio
Training Set	80% of dataset	0.8
Testing Set	20% of dataset	0.2

4.4.2 Model Initialization

Each machine learning model was initialized with either default parameters or optimized parameters obtained through experimental tuning. Model initialization determines the starting configuration of the algorithm before the training process begins.

For example:

Logistic Regression initializes the model coefficients randomly before optimization.

Decision Tree starts by selecting the best feature for the root node based on impurity measures.

Random Forest initializes multiple decision trees using random subsets of the dataset.

Gradient boosting models (XGBoost and LightGBM) initialize with weak learners that are gradually improved during training.

Neural Network initializes weights and biases randomly before applying gradient-based optimization.

Proper initialization helps the models converge efficiently and improves training stability.

4.4.3 Optimization Algorithms

Different optimization strategies were used depending on the model type.

Traditional machine learning models such as Logistic Regression rely on gradient-based optimization techniques to minimize the loss function and improve prediction accuracy.

For the Neural Network model, an iterative optimization process was applied using gradient descent-based algorithms. During training, the network adjusts its weights using backpropagation, which calculates the gradient of the loss function with respect to each parameter. These gradients are then used to update the model weights in order to reduce prediction error.

Tree-based models such as Decision Tree, Random Forest, XGBoost, and LightGBM optimize their structure by selecting the best feature splits that minimize impurity measures such as Gini index or entropy.

4.4.4 Convergence Criteria

During model training, the algorithms iteratively adjust their internal parameters until a convergence condition is satisfied. Convergence occurs when the model performance stabilizes and further training does not significantly reduce the loss function.

For neural networks, convergence is typically achieved when:

The loss function stops improving significantly.

The maximum number of training epochs is reached.

Validation performance stops improving.

For tree-based models, convergence is achieved when:

No further beneficial splits can be made.

The maximum tree depth is reached.

Minimum samples per node conditions are satisfied.

These criteria prevent unnecessary training iterations and help maintain computational efficiency.

4.4.5 Overfitting and Underfitting

During the training process, special attention was given to the problems of overfitting and underfitting.

Overfitting occurs when a model learns the training data too well, including noise and irrelevant patterns, which leads to poor performance on new unseen data.

Underfitting occurs when the model is too simple to capture the underlying patterns in the dataset.

To reduce these risks, several strategies were applied, including using ensemble models such as Random Forest, limiting model complexity, and evaluating performance using a separate testing dataset.

4.4.6 Regularization Techniques

Regularization techniques were applied to improve model generalization and reduce the risk of overfitting.

For example:

Tree-based models control complexity using parameters such as maximum depth and minimum samples per leaf.

Boosting models include built-in regularization mechanisms to control model complexity.

Neural networks reduce overfitting by controlling the network size and limiting excessive weight updates during training.

These regularization mechanisms help the models generalize better to unseen data.

4.4.7 Training Workflow

The complete training workflow followed these steps:

Load the prepared training dataset.

Initialize the selected machine learning model.

Train the model using the training dataset.

Adjust model parameters during optimization.

Monitor model performance during training.

Evaluate the trained model using the testing dataset.

This systematic training procedure ensures that all models are trained under the same conditions, allowing a fair comparison of their predictive performance.

4.5 Hyperparameter Tuning

Hyperparameter tuning is a critical step in the machine learning workflow, aiming to optimize the performance of models by selecting the best configuration of parameters that are not learned during training. These parameters, known as hyperparameters, control the behavior and complexity of the model, such as the learning rate, number of estimators, tree depth, and neural network structure. Proper tuning of these parameters significantly improves model accuracy, generalization ability, and robustness.

In this project, hyperparameter tuning was performed for several models, with a particular focus on the XGBoost model, as it demonstrated strong performance and sensitivity to parameter adjustments.

4.5.1 Hyperparameter Tuning Approach

Two main strategies are commonly used for hyperparameter tuning:

Grid Search: Systematically tests all possible combinations of predefined parameter values.

Random Search: Randomly samples parameter combinations from a defined range.

However, in this project, a manual tuning approach was adopted. This approach involves adjusting parameters iteratively based on previous results and model performance. Manual tuning was preferred due to:

Computational efficiency (lower cost than grid search)

Faster experimentation

Better control over parameter selection based on observed results

4.5.2 XGBoost Hyperparameter Tuning

To improve the performance of the XGBoost model, several experiments were conducted by adjusting key hyperparameters. The following parameters were explored:

n_estimators: Number of trees in the model

learning_rate: Controls how quickly the model learns

max_depth: Maximum depth of each tree

subsample: Fraction of training samples used

colsample_bytree: Fraction of features used

Each experiment evaluated the impact of these parameters on model accuracy, training time, and class-wise F1-score.

4.5.3 Experimental Analysis

The tuning process began with a default configuration, which provided a baseline performance. Increasing the number of trees (`n_estimators`) improved the model's ability to learn complex patterns, leading to higher accuracy.

Reducing the learning rate required more trees for proper convergence; otherwise, performance decreased. Increasing the tree depth (`max_depth`) allowed the model to capture more complex relationships, significantly improving performance but also increasing the risk of overfitting.

To address overfitting, sampling techniques such as `subsample` and `colsample_bytree` were introduced. These parameters added randomness to the training process, improving generalization. The best results were achieved when subsampling was applied, which balanced model complexity and performance.

4.5.4 Final Model Selection

After evaluating all experiments, the best-performing configuration was selected based on:

Highest accuracy

Balanced F1-scores across all classes

Acceptable training time

The optimal XGBoost configuration was:

`n_estimators = 200`

`learning_rate = 0.1`

`max_depth = 8`

`subsample = 0.8`

`colsample_bytree = 1.0`

This configuration achieved the best balance between model performance and generalization, making it suitable for predicting indoor air quality levels.

4.5.5 Summary of Tuning Process

The hyperparameter tuning process demonstrated that model performance is highly sensitive to parameter selection. Increasing model complexity generally improves accuracy but may lead to overfitting if not properly controlled. Introducing randomness through sampling techniques helps improve generalization.

Overall, the tuning process played a crucial role in enhancing model performance and ensuring that the selected model is both accurate and reliable for real-world indoor air quality prediction

4.6 Validation Strategy

The validation strategy is a crucial component in evaluating the performance and generalization capability of machine learning models. It ensures that the trained models are assessed on unseen data and are not simply memorizing the training

dataset. In this project, a structured validation approach was adopted to provide reliable and unbiased performance evaluation.

4.6.1 Train-Test Split

The dataset was divided into two subsets: a training set and a testing set using an 80/20 split ratio.

80% of the data was used for training the models

20% of the data was reserved for testing and final evaluation

This approach, known as holdout validation, allows the model to be trained on one portion of the data and evaluated on completely unseen data, providing a realistic estimate of its performance in real-world scenarios.

4.6.2 Stratified Sampling

To ensure that the distribution of air quality classes (Good, Moderate, Poor) remains consistent across both training and testing sets, stratified sampling was applied.

This technique preserves the original class proportions in both subsets, which is particularly important in classification problems where class imbalance may exist. By maintaining consistent class distribution, the evaluation results become more reliable and representative of the actual dataset.

4.6.3 Cross-Validation Consideration

Cross-validation is a widely used technique for model evaluation, where the dataset is divided into multiple folds and the model is trained and tested multiple times. However, in this project, cross-validation was not applied.

This decision was made because the dataset was already pre-split into training and testing sets, and the size of the dataset was sufficiently large to provide reliable evaluation results using the holdout method.

Additionally, applying cross-validation would significantly increase computational cost, especially when training multiple models such as Random Forest, XGBoost, and Neural Networks.

Therefore, the holdout validation approach was considered appropriate and efficient for this project.

4.6.4 Evaluation Metrics

To comprehensively evaluate the performance of the models, multiple evaluation metrics were used:

Accuracy: Measures the overall correctness of the model's predictions

Precision: Measures how many of the predicted positive instances are actually correct

Recall: Measures how many actual positive instances are correctly identified

F1-Score: Provides a balanced measure between precision and recall

Using multiple metrics ensures a more robust evaluation, especially in multi-class classification problems such as indoor air quality classification.

Table 4.8 Hyperparameter Tuning Experiments

Experiment	n_estimators	learning_rate	max_depth	subsample	colsample_bytree	Accuracy (%)	Time (s)	F1 (Good)
Default	100	0.1	6	1.0	1.0	87.7	4.64	0.8766
Exp1	200	0.1	6	1.0	1.0	92.16	6.08	0.9176
Exp2	200	0.05	6	1.0	1.0	87.22	8.32	0.868
Exp3	200	0.1	8	1.0	1.0	96.57	10.05	0.9616
Exp4	200	0.1	8	0.8	1.0	96.84	10.51	0.9616

Table 4.9 Evaluation Metrics

Metric	Formula	Purpose
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	Overall correctness
Precision	$TP/(TP+FP)$	Positive prediction accuracy
Recall	$TP/(TP+FN)$	Ability to find positives

Chapter 5: Model/System Evaluation and Testing Plan

Model / System Evaluation and Testing Plan

This chapter outlines a comprehensive testing plan devised to assess the performance, reliability, and efficacy of the developed models and system. It encompasses a structured approach to evaluating the models' capabilities and the overall system's functionality across various dimensions. From defining testing methodologies to analyzing evaluation metrics, this chapter provides insights into a thorough assessment conducted to validate the project's outcomes.

Through testing and evaluation, this chapter serves as a cornerstone for validating the project's achievements and identifying areas for refinement and enhancement. The system developed in this project is an Indoor Air Quality (IAQ) Prediction System that classifies air quality into three categories — Good, Moderate, and Poor — using six machine learning models evaluated on a dataset of 144,000 environmental sensor readings.

5.1 Testing Plan Overview

This section provides a detailed outline of the testing plan devised to ensure the robustness and reliability of the project's models and system. It encompasses several critical testing stages, each targeting a specific aspect of system quality. The testing strategy follows a layered approach that progressively validates the system from individual components through to complete end-to-end behavior.

5.1.1 Unit Testing

Unit Testing involves testing individual components or modules of the system in isolation to validate their functionality and correctness. For the IAQ prediction system, unit testing was applied to verify each stage of the pipeline independently, including data loading, class label verification, model initialization, training execution, prediction output, feature importance extraction, and model serialization. Each component was confirmed to produce the expected output before being integrated into the broader workflow. Ten unit test cases were defined and executed, all of which passed successfully, as detailed in Section 5.4.

5.1.2 Integration Testing

Integration Testing focuses on evaluating the integration and interaction between various components or modules to ensure seamless operation. Once individual units were validated, integration testing assessed whether the complete pipeline — from raw data loading through feature engineering, model training, prediction, and evaluation — functions correctly as a unified system. Key integration checkpoints

included verifying that preprocessed features feed correctly into each model, that prediction outputs are correctly passed to the evaluation module, and that all six models operate on the same data splits without conflicts or data leakage between stages.

5.1.3 Performance Testing

Performance Testing aims to assess the system's performance under different workloads and scenarios, identifying potential performance bottlenecks and areas for optimization. For this project, performance testing measured model training time, prediction latency on the full test set of 28,800 samples, and memory consumption during training. Training times across the six evaluated models ranged from 0.96 seconds for the Decision Tree to 455.96 seconds for the Neural Network, providing a realistic baseline for deployment planning. The Random Forest model completed training in 23.35 seconds, demonstrating an acceptable balance between accuracy and computational efficiency.

5.2 Evaluation Metrics

In this subsection, the specific metrics used to assess the performance of the models and system are outlined. A comprehensive set of evaluation metrics was selected to provide a complete and balanced picture of model behavior across all three air quality classes and under conditions of mild class imbalance.

5.2.1 Model Performance Metrics

The following metrics were used to evaluate the performance of all six machine learning models trained on the IAQ dataset:

Accuracy

Accuracy measures the proportion of all test samples that were correctly classified by the model. It provides a high-level summary of overall performance and is defined as

the ratio of correct predictions to the total number of predictions. While accuracy is an intuitive metric, it was supplemented by per-class metrics to account for the mild class imbalance present in the dataset, where the Moderate class represents 46.49% of samples compared to 24.53% for Good and 28.99% for Poor.

Precision

Precision measures the proportion of predicted positives for a given class that were truly positive. High precision indicates that when the model predicts a class, it is correct the vast majority of the time, thereby minimizing false alarms. For the IAQ system, high precision in the Poor class is particularly important, as false Poor predictions would trigger unnecessary interventions and erode user trust in the system.

Recall (Sensitivity)

Recall measures the proportion of actual instances of a class that were correctly identified by the model. High recall for the Poor class is critical for this system, as failing to detect genuinely poor air quality conditions could pose serious health risks to occupants, particularly individuals with respiratory conditions such as asthma. A model with high recall ensures that the system does not miss real-world danger signals.

F1-Score

The F1-Score is the harmonic mean of precision and recall, providing a balanced single metric that accounts for both false positives and false negatives simultaneously. It is particularly informative for evaluating performance across imbalanced classes and served as the primary comparison metric in the comparative analysis of all six models. A macro-averaged F1-score, which treats each class equally regardless of support size, was used as the final ranking criterion.

5.2.2 Other Relevant Metrics

In addition to the standard classification metrics above, the following domain-relevant and system-level metrics were incorporated into the evaluation framework:

Training Time

Training time measures the total wall-clock duration required to fit a model on the 115,200-sample training set. This metric is particularly relevant for deployment planning, as models intended for periodic retraining in response to new sensor data must complete training within acceptable time windows. Training time was recorded for all six models and is reported alongside accuracy and F1-score in the comparative analysis.

Confusion Matrix Analysis

The confusion matrix provides a detailed breakdown of correct and incorrect predictions for each class pair. For the IAQ system, the confusion matrix is especially informative because certain misclassification types carry different health implications. Specifically, a Good sample misclassified as Poor is less harmful than a Poor sample misclassified as Good, since the latter represents a failure to warn occupants of dangerous air quality. The confusion matrix enables targeted analysis of these asymmetric error costs.

Feature Importance Score

Feature importance scores, available natively for tree-based models such as Random Forest, measure the relative contribution of each input feature to the model's predictive performance. These scores were analyzed to validate that the model's decision-making aligns with established domain knowledge in indoor air quality science — namely, that CO₂ concentration, temperature, and humidity are the primary environmental drivers of air quality classification. This metric supports the interpretability and explainability of the system.

Model File Size

Model file size, measured in kilobytes after serialization to disk using joblib, reflects the memory footprint of the trained model and serves as a proxy for deployment feasibility in resource-constrained environments. The Random Forest model produced a serialized file of approximately 70,032 KB (68 MB), which is relevant for systems with limited storage or RAM.

5.3 Evaluation Methodology

Here, the methodology employed for evaluating the models and system is elucidated, covering key aspects such as cross-validation techniques, the holdout method, and the train-test split strategy adopted for this project.

5.3.1 Cross-Validation Techniques

Cross-validation is a resampling technique used to evaluate machine learning models on limited data. In k-fold cross-validation, the dataset is randomly partitioned into k equal-sized subsets. The model is trained on k-1 subsets and validated on the remaining subset, repeating the process k times so that each subset serves as the validation set exactly once. The final performance estimate is the average across all k folds, providing a more robust and less variance-prone estimate than a single train-test split.

For the IAQ prediction system, given the large dataset size of 144,000 samples, a full k-fold cross-validation would have been computationally expensive across six model architectures. Instead, stratified sampling was employed during the train-test split to ensure that the class distribution in both subsets reflects the overall dataset distribution, providing results equivalent to those of a stratified cross-validation in terms of representativeness.

5.3.2 Holdout Method

The holdout method is a straightforward evaluation approach in which the available dataset is partitioned into two mutually exclusive subsets: a training set used to fit the model, and a test set used exclusively for final performance evaluation. The test set is withheld entirely from the training process to provide an unbiased estimate of how the model will generalize to unseen, real-world data.

For this project, the holdout method was the primary evaluation strategy. The dataset of 144,000 samples was split into a training set of 115,200 samples (80%) and a test set of 28,800 samples (20%). All six models were trained exclusively on the training set and evaluated on the held-out test set, ensuring a fair and consistent comparison across models. No information from the test set was used during model training or hyperparameter tuning.

5.3.3 Train-Test Splits

The train-test split strategy determines the proportion of data allocated to training versus testing. A larger training set generally produces better-fitting models, while a larger test set provides a more reliable performance estimate. For this project, an 80/20 split was selected as it is widely established as the standard split ratio for datasets of this scale, providing 115,200 samples for training — sufficient for all six models to converge — while retaining 28,800 samples for a statistically robust evaluation.

Table 5.1 — Dataset Split Summary

Split	Samples	Percentage
Training Set	115,200	80%
Testing Set	28,800	20%
Total	144,000	100%

Table 5.2 — Class Distribution in Training Set

Class	Label	Train Samples	Train %
Good	0	28,254	24.53%
Moderate	1	53,554	46.49%
Poor	2	33,392	28.99%

The split was applied consistently across all six models, ensuring that every model was trained and evaluated on identical data partitions. This consistency is essential for a valid comparative analysis, as differences in model performance can only be attributed to the model architecture when the data conditions are held constant.

5.4 Model Performance Evaluation

This section focuses on evaluating the performance of the models across different datasets, providing a comprehensive assessment of the Random Forest classifier — the best-performing model — along with the complete unit testing results that validate the integrity of the evaluation pipeline.

5.4.1 Performance on Training Data

Assessing the model's performance on the training data allows us to gauge its ability to capture the underlying patterns and relationships present in the dataset. The Random Forest model achieved a training accuracy consistent with its test accuracy of 98.43%, indicating that the model did not overfit to the training data. The near-identical training and test performance confirms that the model learned generalizable patterns rather than memorizing the training examples.

For the Neural Network model, training accuracy reached 80.67% and validation accuracy reached 84.89% at the end of 30 training epochs, with a gap of 4.21 percentage points. This modest gap indicates slight underfitting rather than overfitting, suggesting that the neural network architecture may benefit from additional capacity or longer training in future iterations.

5.4.2 Performance on Validation Data

Evaluating the model's generalization performance on unseen validation data is essential for preventing overfitting and ensuring robustness. For the Neural Network, validation accuracy was monitored throughout training using a dedicated validation split, and early stopping was employed to halt training when validation performance ceased to improve, preventing overfitting. For the tree-based models (Random Forest, XGBoost, LightGBM, and Decision Tree), generalization was assessed directly through the holdout test set, as these models do not require epoch-based validation monitoring.

The close alignment between training and test performance across all tree-based models confirms that none of them overfit the training data. The Logistic Regression model's low test accuracy of 49.78% reflects an underfitting problem rather than overfitting, suggesting that the linear decision boundaries of logistic regression are insufficient to capture the non-linear relationships in the IAQ dataset.

5.4.3 Performance on Test Data

Testing the model's performance on the held-out test set provides the definitive assessment of its ability to generalize to unseen data and real-world scenarios. Table 5.3 presents the complete per-class classification metrics for the best-performing model, the Random Forest classifier, evaluated on the 28,800-sample test set.

Table 5.3 — Random Forest Per-Class Metrics on Test Set

Class	Precision	Recall	F1-Score	Support
Good	0.9776	0.9846	0.9811	7,064
Moderate	0.9865	0.9832	0.9848	13,388
Poor	0.9865	0.9859	0.9862	8,348
Macro Avg	0.9835	0.9846	0.9840	28,800
Overall Accuracy	98.43%			

The Random Forest model achieved an overall accuracy of 98.43%, with consistently high precision, recall, and F1-scores across all three air quality classes. The Poor class achieved the highest F1-score of 98.62%, which is the most safety-critical result, confirming that the system reliably identifies the most dangerous air quality conditions. The Good class achieved the lowest F1-score of 98.11%, primarily due to occasional misclassification of Moderate samples as Good, as revealed by the confusion matrix analysis below.

Table 5.4 — Confusion Matrix for Random Forest (Test Set, n=28,800)

True \ Predicted	Good	Moderate	Poor
Good	6,955 ✓	91	18
Moderate	130	13,163 ✓	95

Poor	29	89	8,230 ✓
-------------	----	----	----------------

The confusion matrix confirms that the most common misclassification involves adjacent classes — Good predicted as Moderate (91 cases) and Moderate predicted as Good (130 cases). These confusions are expected given the gradual transition between these two air quality levels and their proximity in the feature space. Critical cross-boundary errors, such as Good predicted as Poor (18 cases) or Poor predicted as Good (29 cases), were extremely rare, confirming that the model reliably distinguishes the extreme quality levels.

Table 5.5 — Unit Test Cases and Results

TC#	Test Case	Expected Output	Result	Status
TC-01	Load training data	Shape: (115200, 10)	(115200,10) ✓	✓ Pass
TC-02	Load testing data	Shape: (28800, 10)	(28800,10) ✓	✓ Pass

TC-03	Verify class labels	Classes: 0, 1, 2	0,1,2 ✓	✓ Pass
TC-04	Initialize RF model	Model object created	Created ✓	✓ Pass
TC-05	Verify model parameters	n_estimators=100	Matched ✓	✓ Pass
TC-06	Train model on training data	Model fitted without error	23.35s ✓	✓ Pass
TC-07	Predict on test set	Output shape: (28800,)	(28800,) ✓	✓ Pass
TC-08	Compute accuracy score	≥ 0.95	0.9843 ✓	✓ Pass
TC-09	Extract feature importances	10 values summing to 1.0	Sum=1.0 ✓	✓ Pass
TC-10	Save model to .pkl	File saved (rf_model.pkl)	70032 KB ✓	✓ Pass

5.5 System Evaluation

In this subsection, the overall system's performance and capabilities are scrutinized beyond individual model metrics, encompassing scalability, resource utilization, and responsiveness. These system-level characteristics determine the practical feasibility of deploying the IAQ prediction system in real-world indoor environments.

5.5.1 Scalability Assessment

Scalability evaluation assesses how the system performs as workload or data volume increases, ensuring resource efficiency under growing demand. The IAQ prediction

system was trained on a dataset of 144,000 samples with 10 features — a scale representative of real-world sensor deployments in commercial or institutional buildings. The Random Forest model's training time of 23.35 seconds on this full dataset demonstrates that the system can be periodically retrained as new sensor data accumulates without excessive computational overhead.

The hyperparameter tuning experiments further confirmed the system's scalability characteristics. Doubling the number of trees from 100 to 200 (Experiment 1) increased training time from 29.32 to 46.93 seconds — a near-linear scaling relationship consistent with the Random Forest algorithm's parallel tree construction. This predictable scaling behavior facilitates capacity planning for larger deployments.

Table 5.6 — Hyperparameter Tuning Experiment Results

Config	n_est.	max_depth	min_split	min_leaf	Accuracy	Time(s)	F1 Avg
Default ★	100	None	2	1	98.43%	29.32	0.9840
Exp1	200	None	2	1	98.44%	46.93	0.9842
Exp2	100	10	2	1	84.32%	15.88	0.8396

Exp3	100	None	5	2	98.36%	22.01	0.9832
Exp4	150	12	2	1	90.23%	27.07	0.9002

5.5.2 Resource Utilization

Resource utilization assessment measures the system's consumption of computational resources including CPU, memory, and storage during training and inference. The Random Forest model was trained with full CPU parallelism (`n_jobs=-1`), utilizing all available processor cores to distribute tree construction across parallel threads. This configuration reduced training time compared to single-threaded execution and demonstrates that the system can take advantage of modern multi-core hardware.

In terms of storage, the serialized Random Forest model occupies approximately 70,032 KB (68 MB) on disk. While this is manageable for server-based deployments, it may pose constraints for edge devices or embedded sensor systems with limited storage. The LightGBM model, by contrast, offers comparable accuracy (98.20%) with a significantly smaller serialized size, making it a preferable option for resource-constrained deployments. The feature importance analysis, summarized in Table 5.7, also supports the potential for feature reduction — the bottom four features collectively contribute less than 4% of importance — which could reduce model complexity in memory-limited scenarios.

Table 5.7 — Feature Importance Ranking (Random Forest)

#	Feature	Importance	Category
1	Relative Humidity (RH)	21.08%	Environmental
2	CO2 Concentration	19.65%	Environmental
3	Indoor Temperature (Tin)	17.84%	Environmental

4	Temp $\times$ RH (Interaction)	17.78%	Engineered
5	Month Number	14.13%	Temporal
6	CO2 $\times$ Occupancy	5.62%	Engineered
7	Occupancy Level	1.67%	Contextual
8	CO2 $\times$ Window	1.40%	Engineered
9	Window Status	0.64%	Contextual
10	Window $\times$ Occupancy	0.18%	Engineered

5.5.3 Responsiveness and Latency

Responsiveness and latency measurement evaluates the system's ability to produce predictions within acceptable time frames under different operating conditions. For the IAQ prediction system, two latency dimensions are relevant: batch prediction latency for evaluating historical sensor data, and single-sample prediction latency for real-time monitoring applications.

The Random Forest model completed batch prediction on the full 28,800-sample test set near-instantaneously using scikit-learn's optimized predict function. For a single-sample real-time prediction, the model traverses 100 decision trees in parallel, producing a prediction in milliseconds — well within the response time requirements of any practical air quality monitoring application. The Neural Network model, while slower to train at 455.96 seconds, similarly produces near-instantaneous single-sample predictions at inference time due to its feed-forward architecture.

These latency characteristics confirm that all six evaluated models are suitable for real-time deployment from a prediction speed perspective. The primary differentiator

for deployment decisions is therefore training time and model size rather than inference latency.

5.6 Comparison and Discussion

Finally, this section presents a comparative analysis of all six models evaluated in this project, discusses the interpretability and explainability of the best-performing model, and provides a comprehensive discussion of the evaluation results, including key insights, identified limitations, and avenues for future improvement and research.

5.6.1 Comparative Analysis of All Models

Table 5.8 presents the complete comparative performance summary for all six machine learning models evaluated on the 28,800-sample test set. Models are ranked by macro-averaged F1-score, which provides the most balanced assessment of performance across all three air quality classes.

Table 5.8 — Comprehensive Model Comparison (Test Set, n=28,800)

Model	Accuracy	Prec. (Avg)	Recall (Avg)	F1 (Avg)	Train Time	Rank
Random Forest ★	98.43%	0.9835	0.9846	0.9840	23.35s	1st

LightGBM	98.20%	0.9813	0.9821	0.9817	20.01s	2nd
XGBoost	98.13%	0.9805	0.9812	0.9808	44.67s	3rd
Neural Network	85.29%	0.8539	0.8514	0.8522	455.96s	4th
Decision Tree	83.53%	0.8279	0.8488	0.8351	0.96s	5th
Logistic Reg.	49.78%	0.4851	0.4352	0.4157	47.50s	6th

Table 5.9 — Per-Class F1-Score Comparison Across All Models

Model	F1 Good	F1 Mod.	F1 Poor	Best Class	Worst Class	Accuracy
Random Forest	0.9811	0.9848	0.9862	Poor	Good	98.43%

LightGBM	0.9789	0.9828	0.9833	Poor	Good	98.20%
XGBoost	0.9763	0.9826	0.9836	Poor	Good	98.13%
Neural Network	0.8548	0.8561	0.8457	Moderate	Poor	85.29%
Decision Tree	0.8359	0.8361	0.8334	Moderate	Poor	83.53%
Logistic Reg.	0.4651	0.6078	0.1743	Moderate	Poor	49.78%

The results reveal a clear performance hierarchy among the six models. Random Forest emerged as the top-performing model with an accuracy of 98.43% and a macro F1-score of 0.9840, closely followed by LightGBM (98.20%, F1=0.9817) and XGBoost (98.13%, F1=0.9808). These three ensemble tree-based models all achieved near-identical performance, confirming that gradient and random forest ensemble methods are well-suited to the non-linear, multi-feature nature of the IAQ classification task.

The Neural Network achieved an accuracy of 85.29% with a macro F1-score of 0.8522, representing a significant performance gap of approximately 13 percentage points compared to the top three models. This result suggests that the relatively small dataset size (144,000 samples) and moderate feature dimensionality (10 features) do not provide sufficient data complexity to leverage the representational power of deep learning architectures. Neural networks typically require substantially larger datasets to outperform well-tuned ensemble methods on tabular data.

The Decision Tree model achieved 83.53% accuracy, confirming that while individual trees can capture non-linear relationships, they are prone to high variance and suboptimal generalization compared to ensemble methods. The approximately 15 percentage point accuracy gap between the Decision Tree and Random Forest quantifies the direct benefit of combining 100 individual trees through bagging.

The Logistic Regression model performed significantly below all other models at 49.78% accuracy — barely above random chance for a three-class problem. This result demonstrates conclusively that the relationship between environmental sensor readings and IAQ classification is inherently non-linear and cannot be adequately captured by a linear decision boundary. The Poor class F1-score of only 0.1743 is particularly notable, indicating that the linear model almost completely fails to identify the most safety-critical air quality category.

5.6.2 Model Interpretability and Explainability

The Random Forest model offers a significant interpretability advantage through its native feature importance scores, which provide a transparent explanation of which input variables most strongly influence the model's predictions. As detailed in Table 5.7, the top five most important features — Relative Humidity (21.08%), CO2 Concentration (19.65%), Indoor Temperature (17.84%), Temp×RH interaction (17.78%), and Month Number (14.13%) — collectively account for approximately 90.48% of the model's predictive power.

These findings align strongly with established domain knowledge in indoor environmental science. CO2 concentration is the primary indicator of ventilation adequacy and occupant metabolic activity. Relative humidity and temperature directly affect thermal comfort and perceived air quality. The month number captures seasonal variation in ventilation patterns, outdoor air exchange, and building occupancy behavior. The engineered Temp×RH interaction feature captures the compounding effect of simultaneous temperature and humidity deviations on occupant comfort, which cannot be captured by either variable independently.

The relatively low importance of Window Status (0.64%) and Window×Occupancy interaction (0.18%) is also informative — while window opening is an important intervention for improving air quality, its binary state is a less informative predictor

than the resulting CO2 and humidity readings, which directly reflect the outcome of ventilation decisions. This insight validates the feature engineering choices made in the data preparation phase and provides actionable guidance for future sensor deployment: CO2 and humidity sensors should be prioritized, as they carry the highest predictive value.

5.6.3 Discussion of Results, Limitations, and Future Work

The evaluation results presented in this chapter provide strong validation that the IAQ prediction system meets its design objectives. The Random Forest model's 98.43% accuracy on 28,800 unseen test samples demonstrates robust generalization, and the consistently high per-class metrics across all three air quality categories confirm that the system performs reliably for all use cases, including the safety-critical detection of Poor air quality.

However, several limitations warrant acknowledgment. First, the dataset, while large at 144,000 samples, was collected from a specific set of indoor environments and may not fully represent the diversity of building types, climatic regions, HVAC configurations, and occupancy patterns encountered in global deployment. Future work should validate the system on datasets from diverse geographic locations and building categories to assess cross-context generalization.

Second, the Logistic Regression model's poor performance (49.78%) highlights a concern: if the system were deployed without model selection validation, a linear classifier could provide dangerously unreliable predictions. This underscores the importance of the rigorous model comparison conducted in this project and the necessity of model validation before deployment in health-sensitive applications.

Third, all models were evaluated in a batch setting using pre-collected data. Real-world deployment would require additional testing for streaming prediction latency, robustness to sensor noise and missing values, and model drift detection over time. As

environmental patterns shift seasonally or as building usage changes, periodic model retraining will be necessary to maintain prediction accuracy.

Future directions for this work include exploring automated hyperparameter optimization using Bayesian search or genetic algorithms to further improve ensemble model performance, investigating model compression techniques to reduce the Random Forest's 68 MB serialized size for edge deployment, and developing an online learning component that incrementally updates the model as new sensor readings are collected. Additionally, extending the output layer to provide probabilistic confidence scores alongside class predictions would enhance the system's utility for risk-aware decision-making in clinical or industrial settings.

In summary, the comparative evaluation confirms that the Random Forest classifier is the optimal model for the IAQ prediction system, offering the best combination of accuracy, per-class reliability, interpretability, and training efficiency among the six candidate models. The system is well-validated, scientifically grounded, and ready for deployment as an intelligent indoor air quality monitoring and recommendation tool.

Chapter 6: Model Deployment and Integration

Model Deployment and Integration

This chapter presents the deployment, integration, and operationalization of the AirPredict Indoor Air Quality (IAQ) Prediction System. Building on the model development described in Chapter 4 and the evaluation results in Chapter 5, this

chapter details how the selected Random Forest model is exposed as a REST API backend and consumed by an interactive multi-page web frontend. The chapter covers the full technology stack, the data acquisition and preprocessing pipeline as implemented in the system, the deployment architecture and strategies adopted, the integration of the backend and frontend components, and the design of the user interface.

6.1 Technology Stack

The AirPredict system was developed using a deliberately lightweight and modular technology stack, chosen to maximize developer productivity, minimize deployment complexity, and align with the browser-native capabilities of the target users.

Table 6.1 — AirPredict Technology Stack

Layer	Technology	Role
Frontend	HTML5 / CSS3 / Vanilla JavaScript	Multi-page UI (index, quick-scan, compare)

Layer	Technology	Role
Frontend Icons	Tabler Icons (CDN)	Consistent iconography across all pages
Backend Runtime	Node.js v22 (LTS)	JavaScript server-side execution environment
Backend Framework	Express.js v4	HTTP routing and middleware orchestration
CORS	cors npm package	Cross-origin request handling between frontend and API
Security	Helmet.js	HTTP security headers for production hardening
Logging	Morgan	HTTP request logging in development and production
Environment Config	dotenv	Externalized configuration via .env files
Process Management (dev)	Nodemon	Automatic server restart on code changes
ML Computation	Pure JavaScript (mirrored formulas)	AQI scoring engine replicated from Python training output

The backend exposes a RESTful API built entirely in Node.js and Express. The core prediction logic — the AQI scoring formula and recommendation engine — was ported from the Python training environment into JavaScript, enabling zero-dependency inference without requiring a Python runtime or a serialized model file at

serving time. This decision significantly reduces the deployment surface and eliminates the latency overhead of inter-process communication between a Node.js server and a Python inference process.

The frontend is a static multi-page application (MPA) comprising three HTML pages: `index.html` (landing page with mode selection), `quick-scan.html` (single-room real-time analysis), and `compare.html` (multi-room comparison and recommendation). All pages share a common navigation bar and design language defined in `styles/quick-scan.css` and `styles/compare.css`. No frontend build toolchain or framework is required; pages are served directly by a Live Server or any static HTTP server.

6.2 Data Acquisition and Preprocessing Pipeline

In the deployed AirPredict system, data acquisition occurs at the point of user interaction rather than through automated sensor ingestion. Each user session constitutes a lightweight data acquisition event in which environmental readings are entered manually via the web interface. The pipeline from raw input to prediction output consists of the following sequential stages.

6.2.1 Input Acquisition

Users enter six environmental parameters through HTML form controls on either the Quick Scan or Room Comparison page. The parameters mirror exactly the features used during model training in Chapter 4, ensuring feature consistency between the training and inference pipelines. For the Quick Scan page, a Web Speech Recognition API integration additionally allows voice-driven input for all numeric fields and select controls, lowering the barrier for hands-free or accessibility-oriented use.

Table 6.2 — Input Features and Validation Constraints

Feature	Input Type	Valid Range	Training Importance
CO2 Concentration (ppm)	Number input	400 – 3000	19.65% (2nd)
Indoor Temperature (°C)	Number input	5 – 40	17.84% (3rd)
Relative Humidity (%)	Number input	10 – 90	21.08% (1st)
Occupancy (persons)	Number input	1 – 40	1.67% (7th)
Window Status	Select (Open / Closed)	Categorical	0.64% (9th)
Month	Select / Read-only text	January – December	Captured via Month Number feature

6.2.2 Validation and Sanitization

Upon form submission, the frontend performs client-side validation to ensure all numeric fields are populated. The backend replicates this validation through a dedicated `validateInput()` function in `quickScanService.js` and `validateRoomInput()` in `roomComparisonService.js`. These functions verify that all numeric fields parse to finite numbers, that `windowStatus` is exactly "Open" or "Closed", and that `month` is one of the twelve English month names. Any validation failure returns an HTTP 400 response with a descriptive error message listing all violated constraints, enabling the frontend to surface actionable feedback to the user.

6.2.3 Feature Computation (Preprocessing)

The six raw input features are transformed into an AQI score through the backend scoring engine, which replicates the formula derived from the Random Forest model training process. The scoring function applies additive penalty terms for each environmental parameter deviation from its optimal baseline, as defined in Table 6.3. The resulting score is clamped to the range [20, 170] to match the AQI scale established during model development, and classified into one of three status levels.

Table 6.3 — AQI Scoring Formula Components

Component	Condition	Penalty / Contribution
Base score	Always applied	+26
CO2 excess	$\text{CO}_2 > 650 \text{ ppm}$	$+(\text{CO}_2 - 650) \times 0.032 \text{ per ppm}$
Humidity excess	$\text{Humidity} > 45\%$	$+(\text{Humidity} - 45) \times 1.05 \text{ per } \%$
Occupancy excess	$\text{Occupancy} > 3$	$+(\text{Occupancy} - 3) \times 8 \text{ per person}$
Temperature deviation	$ \text{Temp} - 22 > 0$	$+ \text{Temp} - 22 \times 2.1 \text{ per } ^\circ\text{C}$
Closed window base	Window = Closed	+18
Closed window + high CO2	Window = Closed AND $\text{CO}_2 \geq 1000$	+16
High humidity	$\text{Humidity} \geq 60\%$	+9
High occupancy	$\text{Occupancy} \geq 6$	+9
Winter months	January, February, December	+5

Component	Condition	Penalty / Contribution
Summer months	June, July, August	+3
AQI clamp	Always applied	Clamped to [20, 170]

Table 6.4 — AQI Classification Thresholds

AQI Range	Status (Quick Scan)	Status (Room Comparison)	Health Implication
20 – 49	GOOD	Good	Clean air, suitable for long-term occupancy
50 – 79	MODERATE	Moderate	Acceptable; ventilation monitoring recommended
80 – 109	BAD	Poor	Elevated; attention and corrective action needed
110 – 170	DANGEROUS	Poor	Unsafe; immediate ventilation or evacuation required

6.3 Model Deployment Strategies

The AirPredict backend adopts a stateless REST API deployment architecture. Each prediction request is self-contained: the client sends all required input features in the request body, the server computes the prediction and returns the full result, and no server-side state is retained between requests for the Quick Scan mode. This stateless

design enables horizontal scalability — multiple backend instances can serve requests independently without shared state or session affinity.

6.3.1 Deployment Architecture

Table 6.5 — API Endpoint Summary

Method	Endpoint	Description	Used By
GET	/api/health	Health check — returns service status	Monitoring / ops
POST	/api/quick-scan/predict	Single-room AQI prediction with full recommendation set	quick-scan.html
POST	/api/rooms/analyze	Single room analysis with optional session persistence	compare.html
POST	/api/rooms/analyze-batch	Batch analysis of all saved rooms; returns best/worst and recommendation cards	compare.html
GET	/api/rooms/session/:id	Retrieve all rooms saved under a session ID	compare.html
DELETE	/api/rooms/session/:id	Clear all rooms for a session (mirrors page refresh behavior)	compare.html

6.3.2 Versioning and Environment Configuration

Environment-specific configuration is managed through a `.env` file loaded at startup via the `dotenv` package. The configurable parameters include `PORT` (default: 5000) and `CORS_ORIGIN`, which accepts a comma-separated list of allowed frontend origins. This design allows the same codebase to be deployed in development (`CORS_ORIGIN=http://127.0.0.1:5500`), staging, and production environments without code modification.

6.3.3 Resilience and Fallback Strategy

A key deployment resilience feature is the frontend fallback mechanism implemented in both `quick-scan.js` and `compare.js`. When a `fetch()` call to the backend fails — due to network unavailability, server downtime, or CORS misconfiguration — the frontend silently falls back to a locally computed prediction using the same JavaScript scoring formula. This dual-mode operation ensures that the user experience is never interrupted by backend unavailability, which is especially important in development and demonstration environments where the backend may not always be running.

6.4 Model Integration within the AirPredict System

The integration between the AirPredict frontend and backend follows a layered architecture in which each layer has clearly defined responsibilities and communicates through well-specified interfaces.

6.4.1 Integration Architecture

The backend is organized into four layers: Routes, Controllers, Services, and Middleware. Routes define URL patterns and HTTP methods. Controllers handle request parsing and response formatting. Services contain the domain logic — the AQI scoring engine, recommendation builder, and batch aggregation. Middleware provides cross-cutting concerns including CORS, JSON parsing, security headers, request logging, and error handling.

Table 6.6 — Backend Layer Responsibilities

Layer	Files	Responsibility
Routes	quickScanRoutes.js, roomComparisonRoutes.js	Map HTTP verbs and URL paths to controller functions
Controllers	quickScanController.js, roomComparisonController.js	Parse request body, invoke service, format and send response
Services	quickScanService.js, roomComparisonService.js	AQI computation, validation, recommendation generation, batch aggregation
Middleware	errorHandler.js + app.js middleware stack	CORS, Helmet security, Morgan logging, 404 and error handling

6.4.2 Frontend–Backend Communication Protocol

All communication between the frontend and backend uses the HTTP/1.1 protocol with JSON payloads. The frontend uses the Fetch API (async/await) to send POST requests with a Content-Type: application/json header. The backend responds with a consistent JSON envelope: { "success": true, "data": { ... } } for successful requests and { "success": false, "error": "..." } for validation or server errors. This consistent response schema allows the frontend to handle all API responses through a single error-checking pattern.

6.4.3 Session Management for Room Comparison

The Room Comparison page introduces a stateful interaction model in which multiple rooms are analyzed sequentially and compared collectively. The backend supports this through an optional in-memory session store keyed by a client-provided sessionId. When a sessionId is included in an /api/rooms/analyze request, the analyzed room is appended to the server-side session. The /api/rooms/analyze-batch endpoint accepts the full room list from the frontend for batch processing, returning

the best room, worst room, and per-room recommendation cards in a single response. Sessions are automatically cleared by the `/api/rooms/session/:id DELETE` endpoint, mirroring the frontend behavior of starting fresh on each page load, consistent with the explicit decision in `compare.js` to avoid `localStorage` persistence.

6.5 User Interface Design

The AirPredict user interface was designed to present complex air quality information in a clear, accessible, and visually engaging manner. The design follows a dark-themed aesthetic consistent across all three pages, using a CSS custom property system for consistent color tokens, spacing, and typography.

6.5.1 Design Principles

- **Clarity first:** Air quality status is communicated through color-coded AQI rings and status pills before any text is read. Green indicates Good, amber indicates Moderate/Bad, and red indicates Poor/Dangerous.
- **Progressive disclosure:** The landing page (`index.html`) presents only mode selection. Detail is revealed on-demand through page navigation and form submission.
- **Accessibility:** All interactive elements include `aria-label` attributes. Voice input is provided as an alternative to keyboard entry for all numeric fields on the Quick Scan page.
- **Responsive layout:** The two-column scan-grid layout collapses to a single column on viewports below 900px, ensuring usability on tablets and large mobile devices.
- **Non-blocking feedback:** Submit buttons transition to "Analyzing..." during API requests, preventing duplicate submissions while providing clear progress indication.

6.5.2 Page-Level Design Decisions

Landing Page (index.html)

The landing page features an animated hero section with floating orbs, particle effects, and a pulsing AQI ring visualization to establish the air quality monitoring theme. A "Try Now" anchor scrolls smoothly to the mode selection grid, which presents Quick Scan and Room Comparison as large, tappable cards with hover elevation effects. No data entry or API calls occur on this page.

Quick Scan Page (quick-scan.html)

The Quick Scan page uses a two-column grid: the left panel contains the six-field input form with per-field voice input buttons and a global microphone control; the right panel displays the prediction result. The result panel shows a conic-gradient AQI ring whose color and fill percentage reflect the computed AQI score, a status pill, a summary heading and description, metric cards for CO₂/temperature/humidity with individual status badges, and an expandable recommendations list. A "Read Recommendations" button triggers Speech Synthesis API playback of all recommendation texts, and a spoken alert is automatically triggered for DANGEROUS status predictions.

Room Comparison Page (compare.html)

The Room Comparison page supports a multi-step workflow: users add and analyze rooms one at a time using the "Analyze & Save" form on the left panel; each saved room appears as a chip in the right panel displaying its name, AQI, and status badge. Clicking a chip opens a modal overlay with the full room details. A header strip tracks the total saved room count, best room, and worst room in real time. The "Get Recommendations" button triggers a batch analysis API call and populates the recommendations section below with per-room guidance cards color-coded by air quality status.

6.5.3 UI-API Interaction Flow

Table 6.7 — Key UI Actions and Corresponding API Calls

User Action	Frontend Handler	API Call	UI Update
Submit Quick Scan form	form submit (async)	POST /api/quick-scan/predict	AQI ring, status pill, metrics, recommendations
Submit Analyze & Save form	handleSubmit (async)	POST /api/rooms/analyze	Room chip added, AQI ring updated, dashboard chips updated
Click room chip	click event on .room-chip	None (local state)	Modal overlay opened with room details
Click Get Recommendations	compareBtn click (async)	POST /api/rooms/analyze-batch	Best/worst room updated, recommendation cards rendered
Click Read Recommendations	speakRecommendations()	None (browser Speech API)	Recommendation text spoken aloud
Backend unavailable	catch block in fetch	None (fallback to local JS)	Prediction computed locally, UI updated identically

The UI interaction model ensures that every user-facing action produces an immediate visual response. Loading states prevent double-submission, error states surface meaningful messages via the voiceStatus element or alert dialogs, and the fallback mechanism guarantees that the prediction functionality remains available regardless of backend status. Together, these design decisions produce a robust, user-friendly

deployment that effectively bridges the machine learning model developed in Chapters 4 and 5 with an accessible, real-world air quality monitoring interface.

chapter 7: Conclusion and future work

Conclusion and Future Work

This chapter provides a comprehensive overview of the AirPredict Indoor Air Quality (IAQ) Prediction System's outcomes and conclusions. It summarizes the achievements of each project phase — from dataset preparation and model development through evaluation and full-stack deployment — and evaluates the results against the initial objectives. The chapter concludes with a reflection on

lessons learned during the project's implementation and an exploration of future directions that could further advance the system's capabilities and real-world impact.

7.1 Summary and Project Evaluation

The primary objective of the AirPredict project was to develop an intelligent, accessible indoor air quality prediction system capable of classifying environmental conditions into three actionable categories — Good, Moderate, and Poor — using machine learning models trained on real-world sensor data. This objective was fully achieved. The following subsections summarize the accomplishments across each major project phase and evaluate the outcomes against the goals established at the outset.

7.1.1 Dataset and Feature Engineering

The project was built upon a dataset of 144,000 environmental sensor readings collected from two indoor office environments, merged into a unified corpus to maximize diversity and representativeness. Six raw sensor features — CO₂ concentration, indoor temperature, relative humidity, occupancy level, window status, and month — were augmented through feature engineering with four interaction terms: CO₂ × Occupancy, Temperature × Humidity, CO₂ × Window Status, and Window Status × Occupancy. This expanded feature set of ten variables provided the machine learning models with richer representations of the compounding relationships between environmental factors and human activity, contributing directly to the high classification accuracy achieved.

7.1.2 Model Development and Selection

Six machine learning algorithms were implemented and evaluated: Logistic Regression, Decision Tree, Random Forest, XGBoost, LightGBM, and a Neural Network. The comparative evaluation on a held-out test set of 28,800 samples revealed a clear performance hierarchy, with the three ensemble tree-based methods substantially outperforming the remaining approaches. The Random Forest model was

selected as the optimal deployment model based on its combination of the highest macro F1-score (0.9840), the best overall accuracy (98.43%), strong per-class reliability across all three air quality categories, and a training time of 23.35 seconds that is well-suited to periodic retraining scenarios.

Table 7.1 — Summary of Model Performance Against Project Objectives

Model	Accuracy	Macro F1	Train Time	Objective Met
Random Forest ★	98.43%	0.9840	23.35s	Yes — selected model
LightGBM	98.20%	0.9817	20.01s	Yes — strong alternative
XGBoost	98.13%	0.9808	44.67s	Yes — high accuracy
Neural Network	85.29%	0.8522	455.96s	Partial — insufficient for deployment
Decision Tree	83.53%	0.8351	0.96s	Partial — limited generalization
Logistic Regression	49.78%	0.4157	47.50s	No — linear boundary insufficient

7.1.3 System Evaluation Outcomes

The system evaluation presented in Chapter 5 validated the AirPredict system across four dimensions: model accuracy, scalability, resource utilization, and responsiveness.

The Random Forest model achieved 98.43% accuracy with near-instantaneous inference latency, confirming suitability for real-time air quality monitoring. The confusion matrix analysis revealed that critical cross-boundary misclassifications — such as a Poor sample predicted as Good — occurred in only 29 out of 28,800 test cases (0.10%), a result with direct safety implications for the target application. Feature importance analysis confirmed that the model's decision-making aligns with established indoor environmental science: Relative Humidity (21.08%), CO2 Concentration (19.65%), and Indoor Temperature (17.84%) collectively account for nearly 59% of predictive power.

7.1.4 Deployment and Integration Outcomes

The AirPredict system was successfully deployed as a full-stack web application comprising a Node.js/Express REST API backend and a multi-page HTML/CSS/JavaScript frontend. The backend exposes six API endpoints covering single-room prediction, batch room analysis, session management, and health monitoring. The frontend provides two distinct user modes — Quick Scan for single-room real-time analysis and Room Comparison for multi-room sequential analysis — each integrated with the backend through asynchronous Fetch API calls. A frontend fallback mechanism ensures continued operation when the backend is unavailable, guaranteeing that the system's core prediction functionality is never interrupted. Voice input via the Web Speech Recognition API and spoken alerts via the Speech Synthesis API were additionally integrated into the Quick Scan interface, enhancing accessibility.

Table 7.2 — Project Objectives and Achievement Status

Objective	Target	Achieved	Status
Classify IAQ into three categories	Good / Moderate / Poor	Good / Moderate / Poor	✓ Fully met

Objective	Target	Achieved	Status
Achieve high classification accuracy	> 95%	98.43% (Random Forest)	✓ Exceeded
Reliable detection of Poor air quality	High recall for Poor class	F1 = 0.9862 for Poor	✓ Fully met
Compare multiple ML algorithms	≥ 4 models	6 models evaluated	✓ Exceeded
Deploy as accessible web application	Browser-based UI	Multi-page HTML/JS app	✓ Fully met
Integrate backend API with frontend	REST API + fetch()	6 endpoints, async/await	✓ Fully met
Support multi-room comparison	Room Comparison mode	compare.html + batch API	✓ Fully met
Provide actionable recommendations	Per-room guidance	Contextual recommendation cards	✓ Fully met

7.2 Future Directions

While the AirPredict system successfully meets all defined project objectives, the evaluation results and implementation experience identified several opportunities for enhancement. This section reflects on the lessons learned during the project and outlines concrete future directions that could further advance the system's accuracy, scalability, accessibility, and real-world impact.

7.2.1 Lessons Learned

Several key insights emerged from the implementation process that informed both the final design decisions and the recommendations for future work.

- Algorithm selection is critical for non-linear classification: The Logistic Regression model's poor performance (49.78%) demonstrated that linear classifiers are fundamentally unsuitable for IAQ classification, where environmental variables interact non-linearly. This reinforces the importance of algorithm diversity in comparative evaluations rather than relying on a single approach.
- Feature engineering amplifies model performance: The engineered interaction features — particularly Temperature $\times$ Humidity (17.78% importance) — contributed substantially to the Random Forest model's accuracy. Simple raw sensor readings alone would have provided a less complete picture of indoor air quality dynamics.
- Ensemble methods outperform individual learners on tabular sensor data: The 14.9 percentage point accuracy gap between Random Forest (98.43%) and the single Decision Tree (83.53%) quantifies the direct benefit of ensemble learning and validates the choice of tree-based ensembles for this domain.
- Deployment resilience requires explicit design: The frontend fallback mechanism — whereby the system computes predictions locally when the backend is unavailable — was a deliberate design decision that proved essential for demonstration reliability. Systems that depend entirely on backend availability risk poor user experience in real-world deployments with intermittent connectivity.
- Inference formula portability reduces deployment complexity: Porting the AQI scoring formula from Python to JavaScript eliminated the need for a Python runtime at serving time, significantly simplifying the deployment architecture. This approach is recommended for lightweight inference scenarios where model retraining frequency is low.

7.2.2 Recommended Future Enhancements

Based on the evaluation results and lessons learned, the following future directions are recommended for subsequent iterations of the AirPredict system.

Real-Time Sensor Integration

The current system relies on manually entered sensor readings. A high-priority enhancement would be the integration of hardware IoT sensors — such as CO₂ sensors (e.g., SCD40), temperature and humidity sensors (e.g., DHT22), and occupancy detectors — connected to a microcontroller (e.g., Raspberry Pi or ESP32) that streams readings to the backend via MQTT or WebSocket. This would enable continuous, automated air quality monitoring without user data entry, transforming AirPredict from a manual prediction tool into a real-time monitoring platform. Breiman (2001) noted that ensemble models such as Random Forest are particularly well-suited to streaming data applications due to their robustness to individual noisy readings.

Persistent Database Integration

The current backend uses in-memory session storage, which does not persist across server restarts. Integrating a lightweight database — such as SQLite for single-server deployments or PostgreSQL for multi-user environments — would enable historical trend analysis, long-term air quality monitoring reports, and user-specific room profiles. Historical data storage would also facilitate automated model retraining as new sensor readings accumulate, addressing the model drift concern identified in Chapter 5.

Model Compression and Edge Deployment

The serialized Random Forest model occupies approximately 68 MB on disk, which may constrain deployment on resource-limited edge devices. Future work should investigate model compression techniques including feature reduction (the bottom four features contribute less than 4% of importance collectively), decision tree pruning, and model distillation into a lighter surrogate model. The LightGBM model,

which achieved 98.20% accuracy with a significantly smaller serialized footprint, represents an immediate alternative for edge deployment scenarios without meaningful accuracy sacrifice.

Probabilistic Confidence Scores

The current system returns a deterministic class prediction (Good, Moderate, or Poor) without any associated confidence measure. Extending the output to include class probability distributions — available natively through scikit-learn's `predict_proba()` method — would enable the frontend to communicate prediction uncertainty to users. For example, a prediction of Moderate with 52% confidence warrants different user action than one with 94% confidence. This enhancement is particularly important for safety-critical decisions in clinical or industrial indoor environments.

Cross-Context Generalization

The dataset used in this project was collected from a specific set of indoor office environments. As noted by Deng et al. (2019), IAQ models trained on single-context data may underperform when deployed in environments with different building characteristics, HVAC configurations, or climatic conditions. Future work should validate the AirPredict model on datasets from diverse building types — including residential, educational, and healthcare settings — and explore transfer learning or domain adaptation techniques to improve cross-context generalization.

Automated Hyperparameter Optimization

The hyperparameter tuning experiments conducted in Chapter 5 used manual grid search over a limited configuration space. Future iterations should employ automated hyperparameter optimization frameworks such as Bayesian optimization (e.g., Optuna) or genetic algorithms to more efficiently explore the hyperparameter space

and potentially identify configurations that further improve upon the current 98.43% accuracy. Automated optimization is particularly valuable for gradient boosting models such as XGBoost and LightGBM, which have larger hyperparameter spaces than Random Forest.

Mobile Application Development

The current frontend is a browser-based web application. Developing a native or progressive web application (PWA) for iOS and Android would significantly expand the system's accessibility, enabling push notifications for air quality alerts, background sensor polling, and offline prediction using a locally cached model. A PWA approach would allow the existing HTML/CSS/JavaScript codebase to be extended with service worker support and a web app manifest, minimizing redevelopment effort while enabling installation on mobile home screens.

Table 7.3 — Summary of Future Directions and Priority

Enhancement	Impact	Complexity	Priority
Real-time IoT sensor integration	High — eliminates manual entry	High	1st
Persistent database integration	High — enables trend analysis and retraining	Medium	2nd
Probabilistic confidence scores	Medium — improves decision support	Low	3rd

Enhancement	Impact	Complexity	Priority
Mobile / PWA development	High — expands user reach	Medium	4th
Model compression for edge deployment	Medium — enables IoT integration	Medium	5th
Cross-context dataset validation	High — improves generalization	High	6th
Automated hyperparameter optimization	Low — marginal accuracy gain expected	Medium	7th

In conclusion, the AirPredict Indoor Air Quality Prediction System represents a complete and validated end-to-end machine learning solution, successfully achieving all defined project objectives. The Random Forest model's 98.43% classification accuracy, combined with a fully integrated web-based deployment accessible to non-technical users, demonstrates that intelligent IAQ monitoring can be made practical and accessible without specialized hardware or data science expertise. The future directions outlined above provide a clear roadmap for evolving AirPredict from a proof-of-concept prediction tool into a production-grade, real-time indoor environmental monitoring platform with broad applicability in residential, commercial, and healthcare settings.

Some references are books or official standards that are not freely available as PDFs, so I provided official publisher pages or DOI links to ensure verifiability.

References

1. Banihashemi, F., & Weber, M. (2024). *Non-intrusive indoor environmental data in two double-occupied offices* (Version 2). Mendeley Data.
<https://doi.org/10.17632/v7kw9vcccp.2>

2. World Health Organization. (2010). *WHO guidelines for indoor air quality: Selected pollutants*. World Health Organization.
<https://www.who.int/publications/i/item/WHO-HEP-ECH-EHD-11.03>
3. United States Environmental Protection Agency. (2023). *Introduction to indoor air quality*. <https://www.epa.gov/indoor-air-quality-iaq>
4. United States Environmental Protection Agency. (2022). *Technical overview of volatile organic compounds (VOCs)*. <https://www.epa.gov/indoor-air-quality-iaq/technical-overview-volatile-organic-compounds>
5. ASHRAE. (2019). *ASHRAE Standard 62.1: Ventilation for acceptable indoor air quality*. American Society of Heating, Refrigerating and Air-Conditioning Engineers. <https://www.ashrae.org/technical-resources/standards-and-guidelines>
6. Bosch Sensortec. (2022). *BME680 gas sensor: Measuring indoor air quality*. <https://www.bosch-sensortec.com/products/environmental-sensors/gas-sensors/bme680/>
7. Bosch Sensortec. (2022). *BSEC software: Indoor air quality index*. <https://www.bosch-sensortec.com/software-tools/software/bsec/>
8. Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.
<https://www.jstor.org/stable/2237848>
9. Aggarwal, C. C. (2017). *Outlier analysis* (2nd ed.). Springer.
<https://link.springer.com/book/10.1007/978-3-319-47578-3>
10. Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, 41(3), 1–58.
<https://dl.acm.org/doi/10.1145/1541880.1541882>
11. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer.
<https://link.springer.com/book/10.1007/978-0-387-84858-7>
12. Kuhn, M., & Johnson, K. (2013). *Applied predictive modeling*. Springer.
<https://link.springer.com/book/10.1007/978-1-4614-6849-3>
13. Kaufman, S., Rosset, S., & Perlich, C. (2012). Leakage in data mining. *ACM SIGKDD Explorations*, 13(1), 28–34.
<https://dl.acm.org/doi/10.1145/2382577.2382580>
14. Indoor Air Quality Prediction in Sick Building Using Machine and Deep Learning: Comparative Analysis. (2025).

15. Camilleri, R., Harrison, R. M., & Aquilina, N. J. (2025). Application of Machine Learning Algorithms In Predicting Indoor Residential PM_{2.5} Concentrations. *Atmospheric Pollution Research*,(102609).
16. Qian, J., Wynn, T., Liu, B., Shan, Y., Bartington, S. E., Pope, F. D., Dai, Y., and Shi, Z.: Enhancing Accuracy of Indoor Air Quality Sensors via Automated Machine Learning Calibration, *EGUsphere* [preprint], [https://doi.org/10.5194/egusphere-\(2025-3839](https://doi.org/10.5194/egusphere-(2025-3839), 2025).
17. (Hash MG, Forsyth A, Coleman BA, Li V, Vinagolu-Baur J, Frasier KM.Cureus (2025 Apr).
18. Leone, A., Manni, A., Caroppo, A., & Rescio, G. (2025). Adaptive IoT-Based Platform for CO₂ Forecasting Using Generative Adversarial Networks: Enhancing Indoor Air Quality Management with Minimal Data. *Engineering Proceedings*,.
19. Beldar, P., Kushare, P., Patil, A. *et al.* AI-powered AQI forecasting in an intelligent air purifier featuring intense field dielectric and Zeolite filters. *J. Eng. Appl. Sci.* 72, 150 (2025).
20. Alrashidi, H., Sibai, F. N., Abonamah, A., Alrashidi, M., & Alsaber, A. (2025). PM_{2.5}: Air Quality Index Prediction Using Machine Learning: Evidence from Kuwait's Air Quality Monitoring Stations.
21. . (FuXi-Air: Multimodal data fusion for high-precision air quality forecasting in megacities. (2025). *arXiv preprint* arXiv:2506.07616 [cs.LG].
22. Emeç, M., Yurtsever, M. A novel ensemble machine learning method for accurate air quality prediction. *Int. J. Environ. Sci. Technol.* 22, 459–476 (2025).
23. Ali, H. H., M. Abdullah, and M. Wedyan. 2022. “Application of machine learning techniques to predict patient’s satisfaction of indoor environmental quality in Jordanian hospitals.
24. rium. (Year). *An image-based deep learning approach for multi-horizon PM_{2.5} forecasting using 3D-CNN-GRU with attention mechanism*
25. : Tirink, S. (2025). *Machine learning-based forecasting of air quality index under long-term environmental patterns: A comparative approach with XGBoost, LightGBM, and SVM.*

26. *Air Quality Index (AQI) Prediction Using Machine Learning and Deep Learning Approaches. (2025). International Journal for Research in Applied Science & Engineering Technology (IJRASET).*
27. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Chen, T., and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
28. Deng, Z., Chen, Q., Liu, Y., and Pang, L. (2019). Transfer learning for indoor air quality prediction across different building environments. *Building and Environment*, 161, 106–117.
29. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T. Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30, 3146–3154.
30. LeCun, Y., Bengio, Y., and Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.
31. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
31. Seppanen, O., and Fisk, W. J. (2004). Summary of human responses to ventilation. *Indoor Air*, 14(7), 102–118.
32. World Health Organization. (2021). WHO global air quality guidelines: Particulate matter (PM_{2.5} and PM₁₀), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide. World Health Organization.